Grado en Ingeniería del Software

# Diseño y Arquitectura Software

## Apuntes Completos de la Asignatura y Guía del Ingeniero de Software

**Autor:**
David

Dpto. de Informática, Lenguajes y Sistemas Informáticos

Universidad Rey Juan Carlos (URJC) | Curso Académico 2025–2026

**Versión del documento:** 1.0
**Compilado:** 9 de noviembre de 2026

**Estado:** Revisado y Completo
**Licencia:** CC BY-SA 4.0

# Índice general


**1. Concepto de Arquitectura Software y Principios de Diseño** **4**
- 1.1. Concepto de Arquitectura Software (AS) . . . 4
  - 1.1.1. Evolución y necesidad de la disciplina . . . 4
  - 1.1.2. La perspectiva de los Stakeholders y las Vistas Arquitectónicas . . . 5
  - 1.1.3. La Definición Formal de Perry y Wolf (1991) . . . 6
  - 1.1.4. La Analogía con la Arquitectura Civil: Puntos Fuertes y Límites . . . 6
  - 1.1.5. El Rol del Arquitecto Software en la Organización . . . 8
  - 1.1.6. Desalineación (*Mismatch*) y Erosión Arquitectónica . . . 8
- 1.2. Modelos de Referencia, Arquitecturas de Referencia y Familias de Productos . . . 9
  - 1.2.1. Distinción entre Modelo de Referencia y Arquitectura de Referencia . . . 9
  - 1.2.2. Caso de Estudio Industrial: AUTOSAR en la Automoción . . . 10
  - 1.2.3. De Sistemas Aislados a Familias de Sistemas (*Software Product Lines*) . . . 11
  - 1.2.4. El Proceso de Personalización y la Gestión de la Variabilidad . . . 11
- 1.3. Principios Fundamentales de Diseño Software . . . 12
  - 1.3.1. Separación de Responsabilidades (*Separation of Concerns - SoC*) . . . 12
  - 1.3.2. Acoplamiento y Cohesión: El Binomio Maestro . . . 12
  - 1.3.3. Anti-patrones de Diseño Estructural: El Anti-patrón God Class . . . 14
  - 1.3.4. Ocultamiento de Información (*Information Hiding*) y Encapsulamiento . . . 15
  - 1.3.5. El Peligro de las Dependencias Circulares y el Principio ADP . . . 15
- 1.4. Inversión de Dependencias (DIP), Inversión de Control (IoC) e Inyección de Dependencias (DI) 16
  - 1.4.1. El Principio de Inversión de Dependencias (DIP) . . . 17
  - 1.4.2. Caso Práctico de Código: Violación y Corrección de DIP . . . 17
  - 1.4.3. Inversión de Control (IoC) y el Principio de Hollywood . . . 19
  - 1.4.4. Inyección de Dependencias (DI): Mecanismos y Roles . . . 19
  - 1.4.5. Modalidades de Inyección de Dependencias . . . 20
  - 1.4.6. Impacto Decisivo en la Testeabilidad: Dobles de Prueba . . . 21
- 1.5. Ejercicios Resueltos y Casos Prácticos de Ingeniería . . . 21
- 1.6. Resumen Ejecutivo y Preguntas de Autoevaluación . . . 25
  - 1.6.1. Batería de Preguntas de Autoevaluación (Estilo Examen URJC) . . . 25

**2. Estilos Arquitectónicos, Patrones de Diseño y Modelado con UML y C4** **27**
- 2.1. Estilos Arquitectónicos frente a Patrones de Diseño: Granularidad e Historia . . . 28
  - 2.1.1. Diferenciación Conceptual y Niveles de Granularidad . . . 28
  - 2.1.2. Evolución Cronológica del Movimiento de Patrones . . . 29
- 2.2. Catálogo Exhaustivo de Estilos Arquitectónicos . . . 29
  - 2.2.1. Tuberías y Filtros (*Pipes and Filters*) . . . 29
  - 2.2.2. Estilo por Capas (*Layered Architecture*) . . . 30
  - 2.2.3. Estilo Cliente-Servidor (*Client-Server*) . . . 30
  - 2.2.4. Modelo-Vista-Controlador (*MVC*) . . . 31

2.2.5. El Estilo REST y Arquitecturas RESTful . . . 31
2.2.6. Arquitecturas Dirigidas por Eventos (*Event-Driven Architecture - EDA*) . . . 32
2.2.7. Otros Estilos Notables: Pizarra (*Blackboard*) y Peer-to-Peer . . . 33
2.3. Patrones Arquitectónicos para Sistemas Distribuidos y Microservicios . . . 33
2.3.1. El Patrón SAGA: Gestión de Transacciones Distribuidas . . . 33
2.3.2. CQRS (*Command Query Responsibility Segregation*) . . . 34
2.3.3. Circuit Breaker (Disyuntor) . . . 34
2.4. Patrones de Diseño Clásicos (GoF) . . . 35
2.4.1. Patrones Creacionales . . . 35
2.4.2. Patrones Estructurales . . . 36
2.4.3. Patrones de Comportamiento . . . 36
2.4.4. Otros Patrones: Parallel Split (Flujos de Trabajo / Workflows) . . . 37
2.5. Vistas Arquitectónicas y Modelado con UML 2.5 . . . 37
2.5.1. El Modelo de Vistas 4+1 de Philippe Kruchten . . . 37
2.5.2. Diagramas de Paquetes: Modelado de Subsistemas y Estilos . . . 37
2.5.3. Taxonomía de Relaciones en Diagramas de Clases UML . . . 37
2.5.4. Mecanismos de Extensión de UML: Estereotipos, Valores Etiquetados y OCL . . . 37
2.6. Notación Arquitectónica Moderna: El Modelo C4 . . . 39
2.7. Ejercicios Resueltos y Casos Prácticos de Examen . . . 39
2.8. Resumen Ejecutivo y Preguntas de Autoevaluación . . . 41
2.8.1. Batería de Preguntas de Autoevaluación . . . 41

**3. Decisiones de Diseño y Gestión del Conocimiento Arquitectónico 42**
3.1. Fundamentos del Conocimiento Arquitectónico y Decisiones de Diseño (ADD) . . . 43
3.1.1. El Cambio de Paradigma: Jan Bosch (2004) . . . 43
3.1.2. Conocimiento Tácito frente a Conocimiento Explícito . . . 43
3.1.3. El Estándar Internacional ISO/IEC/IEEE 42010:2022 . . . 44
3.2. Caracterización de Decisiones y Modelos de Captura . . . 44
3.2.1. Aproximaciones de Captura de Decisiones . . . 44
3.2.2. El Modelo Canónico de Tyree y Akerman (2005) . . . 44
3.3. La Vista de la Decisión y Procesos de AKM . . . 45
3.3.1. La Vista de la Decisión en la Arquitectura . . . 45
3.3.2. Procesos del Ciclo de Vida de AKM . . . 45
3.3.3. Trazabilidad Externa: Forward y Backward Traceability . . . 47
3.3.4. Trazabilidad Interna y Relaciones Ontológicas entre Decisiones . . . 47
3.4. Herramientas de AKM y su Evolución Histórica . . . 48
3.4.1. Herramientas Destacadas . . . 48
3.5. El Estándar Industrial MADR (Markdown Architectural Decision Records) . . . 49
3.5.1. Estructura Oficial de una Plantilla MADR . . . 49
3.5.2. Errores Clave a Evitar en MADR (Examen y Práctica Profesional) . . . 49
3.6. Pensamiento Crítico y Reflexión en la Arquitectura: Modelo Mind 1 / Mind 2 . . . 50
3.7. Ejercicios Resueltos y Casos Prácticos de Examen . . . 51
3.8. Resumen Ejecutivo y Preguntas de Autoevaluación . . . 53
3.8.1. Batería de Preguntas de Autoevaluación . . . 53

**4. Evaluación de Arquitecturas Software y Método ATAM 54**
4.1. El Rol Crítico de los Atributos de Calidad en la Arquitectura . . . 54
4.1.1. Los Requisitos No Funcionales Determinan la Forma del Software . . . 54
4.1.2. La Dificultad Intrínseca de Evaluar Diseños . . . 55
4.1.3. Operacionalización Cuantitativa: De Deseos Vagos a Métricas Medibles . . . 55
4.2. Taxonomías de Atributos de Calidad y Estándar ISO/IEC 25010 . . . 56
4.2.1. Clasificación de Atributos: Runtime, Non-Runtime y Negocio . . . 56
4.2.2. El Modelo de Calidad ISO/IEC 25010:2011 frente a ISO/IEC 25010:2023 . . . 56
4.3. Escenarios de Calidad: Cuantificación Medible de Atributos . . . 58

4.3.1. Las 6 Partes Canónicas de un Escenario (Estándar SEI) . . . . . 58
4.3.2. Ejemplos de Escenarios Canónicos por Atributo . . . . . 59
4.4. El Árbol de Utilidad (*Utility Tree*) . . . . . 59
4.4.1. Estructura Jerárquica del Utility Tree . . . . . 59
4.4.2. Matriz de Priorización de Escenarios ($Importancia, Dificultad$) . . . . . 59
4.5. El Método ATAM (*Architecture Tradeoff Analysis Method*) . . . . . 60
4.5.1. Los Cuatro Hallazgos Cardinales de ATAM . . . . . 60
4.5.2. Roles y Actores en el Proceso ATAM . . . . . 61
4.5.3. Las Fases y los 9 Pasos Secuenciales de ATAM . . . . . 61
4.6. Otros Métodos de Evaluación Arquitectónica . . . . . 62
4.7. Ejercicios Resueltos y Casos Prácticos de Examen . . . . . 62
4.8. Resumen Ejecutivo y Preguntas de Autoevaluación . . . . . 64
4.8.1. Batería de Preguntas de Autoevaluación . . . . . 64

# CAPÍTULO 1

# Concepto de Arquitectura Software y Principios de Diseño

**Objetivos de Aprendizaje del Tema 1**

Al finalizar este capítulo, el estudiante e ingeniero de software será capaz de:

- Comprender qué constituye la **Arquitectura Software (AS)** y diferenciarla con precisión del diseño detallado y de la mera implementación.
- Dominar y aplicar la definición formal de **Perry y Wolf (1991)**: tripleta $\langle \text{componentes}, \text{forma}, \text{lógica} \rangle$.
- Evaluar las analogías estructurales y las divergencias fundamentales entre la ingeniería civil y el software.
- Identificar y mitigar los fenómenos de **desalineación arquitectónica** (*architectural mismatch*) y **erosión arquitectónica** (*architectural drift / erosion*).
- Diferenciar con rigor conceptual entre **Modelos de Referencia**, **Arquitecturas de Referencia** (analizando el caso industrial **AUTOSAR**) y **Familias de Productos Software** (*Software Product Lines*).
- Gestionar la **variabilidad software** (*software variability*) frente a los elementos nucleares comunes (*commonalities*).
- Aplicar de manera sistemática los principios de **Separación de Responsabilidades (SoC)**, **Ocultamiento de Información** (*Information Hiding*), **Alta Cohesión** y **Bajo Acoplamiento**.
- Detectar y refactorizar anti-patrones críticos de diseño como la **God Class** (Blob) y las **dependencias circulares** mediante el principio de dependencias acíclicas (ADP).
- Distinguir, relacionar e implementar sin ambigüedades la tríada: **Dependency Inversion Principle (DIP)**, **Inversion of Control (IoC)** y **Dependency Injection (DI)**, evaluando su impacto directo en la testeabilidad y la mantenibilidad.

## 1.1 Concepto de Arquitectura Software (AS)

### 1.1.1 Evolución y necesidad de la disciplina

Durante las primeras décadas de la informática, el desarrollo de software se centraba fundamentalmente en la resolución algorítmica y la manipulación de estructuras de datos discretas (lo que Niklaus Wirth sintetizó en su célebre obra *Algorithms + Data Structures = Programs*). En aquellos sistemas, el tamaño del

código era reducido y un único programador o un equipo pequeño podía retener en su mente la totalidad del comportamiento del programa.

Sin embargo, a medida que los sistemas crecieron en escala, criticidad y complejidad, la causa principal del fracaso de los proyectos dejó de ser la corrección algorítmica interna para convertirse en la incapacidad de gestionar la **interacción global de componentes heterogéneos**, la integración de subsistemas distribuidos y la satisfacción de atributos de calidad como la disponibilidad, escalabilidad, seguridad y mantenibilidad.

**▶ Concepto Clave: Arquitectura Software (AS)**

La **Arquitectura Software** de un sistema es el conjunto de estructuras requeridas para razonar sobre el sistema, el cual comprende los elementos de software (componentes), las relaciones entre ellos (conectores y topología) y las propiedades visibles de ambos. Representa la versión moderna del diseño de sistemas complejos en el nivel más alto de abstracción.

Una arquitectura software adecuada:

- **Reduce la complejidad cognitiva:** Permite a los seres humanos razonar sobre sistemas masivos ocultando los detalles de implementación de bajo nivel bajo abstracciones bien delimitadas.
- **Canaliza la comunicación entre partes interesadas (*stakeholders*):** Actúa como el lenguaje común entre perfiles dispares: desarrolladores, arquitectos de infraestructura, directores de negocio, especialistas de seguridad y clientes finales.
- **Encarna las decisiones de diseño más tempranas y de mayor coste de cambio:** Cambiar un algoritmo de ordenación en una clase toma minutos; cambiar una arquitectura monolítica a microservicios orientados a eventos puede exigir meses o años de esfuerzo con alto riesgo comercial.

### 1.1.2 La perspectiva de los Stakeholders y las Vistas Arquitectónicas

Un error común entre ingenieros noveles consiste en pensar que la arquitectura software se reduce a un único diagrama en bloque. La realidad es que un sistema complejo no puede representarse de forma inteligible en un único plano:

- Para un **desarrollador**, la arquitectura se visualiza en términos de clases, interfaces, paquetes y patrones de diseño orientados a objetos.
- Para un **administrador de sistemas o ingeniero DevOps**, la arquitectura se visualiza como contenedores, máquinas virtuales, balanceadores de carga, redes virtuales, clústeres de bases de datos y tuberías de despliegue continuo.
- Para el **responsable de negocio o cliente**, la arquitectura es la materialización de los casos de uso, los límites de coste de infraestructura y el cumplimiento normativo (como GDPR o PCI-DSS).

**▲ ¡Atención / Ingeniería!: El modelo de vistas de Kruchten (4+1)**

Para satisfacer a los distintos *stakeholders* sin caer en la confusión, la ingeniería de software estandarizó el uso de **vistas arquitectónicas** (consolidado en el estándar **ISO/IEC/IEEE 42010**):

1. **Vista Lógica:** Satisface los requisitos funcionales. Se enfoca en paquetes, clases, capas y modelos de dominio.
2. **Vista de Procesos:** Analiza la concurrencia, rendimiento, sincronización, hilos de ejecución e integridad de transacciones en tiempo de ejecución.
3. **Vista de Desarrollo (o Implementación):** Modela la organización del código fuente, librerías, dependencias de módulos y artefactos compilados.
4. **Vista Física (o de Despliegue):** Representa la topología de hardware, topología de red, servidores y nube donde corre el software.
5. **Casos de Uso / Escenarios (+1):** Cosan y validan las cuatro vistas anteriores asegurando

que la arquitectura cumpla los objetivos del sistema.

### 1.1.3 La Definición Formal de Perry y Wolf (1991)

En 1991, Dewayne Perry y Alexander Wolf publicaron un artículo seminal titulado *"Foundations for the Study of Software Architecture"*, estableciendo la primera fórmula analítica rigurosa para describir formalmente una arquitectura software:

$$\textbf{AS} = \{\textbf{Elementos (Componentes)}, \textbf{Forma}, \textbf{Lógica / Rationale}\} \tag{1.1}$$

Desglosemos cada uno de los términos de esta tripleta:

1. **Componentes (Elementos Arquitectónicos):**
   Son los bloques constitutivos que encapsulan estado o computación. En lenguajes orientados a objetos y modelado UML se traducen típicamente en clases, paquetes, módulos, bibliotecas o microservicios. Perry y Wolf distinguieron tres subtipos de elementos:
   - *Elementos de Procesamiento:* Transforman datos (ej., un servicio de cálculo de impuestos).
   - *Elementos de Datos:* Almacenan o encapsulan información (ej., un registro, una entidad de base de datos o un DTO).
   - *Elementos de Conexión (Conectores):* Mecanismos que posibilitan la comunicación y la transferencia de control entre elementos (ej., llamadas a función, invocaciones RPC, pasarelas REST, colas de mensajes).
2. **Forma (Form):**
   Define cómo se organizan, interconectan y estructuran espacialmente dichos componentes. Establece las restricciones de topología, la tipología de conexiones permitidas y las propiedades que deben preservar los elementos en su interacción (por ejemplo: "los componentes de la capa de interfaz de usuario sólo pueden invocar a la capa de aplicación, nunca a la capa de acceso a datos de forma directa").
3. **Lógica o Justificación (*Rationale*):**
   Representa el **porqué** de las decisiones de diseño. Documenta la motivación técnica, funcional o de negocio que impulsó la elección de un componente o patrón concreto frente a sus alternativas. Responde a preguntas como: *¿Por qué se eligió una base de datos NoSQL basada en grafos en lugar de una relacional?* o *¿Por qué se seleccionó un patrón Publicador-Suscriptor en lugar de llamadas síncronas HTTP?* Sin la lógica, los futuros ingenieros no pueden evaluar si un cambio en el sistema violará las asunciones originales del diseño.

### 1.1.4 La Analogía con la Arquitectura Civil: Puntos Fuertes y Límites

Desde los inicios de la disciplina, ha sido común establecer un paralelismo entre la construcción de edificios o puentes y el diseño de software. Esta metáfora pedagógica resulta sumamente ilustrativa pero posee límites muy estrictos que todo ingeniero debe conocer:

**▲ ¡Atención / Ingeniería!: ¿Dónde fracasa la analogía civil?**

A diferencia de los puentes o rascacielos:

1. **El software no está sujeto a leyes físicas:** El software es una entidad puramente conceptual e inmaterial. No sufre fatiga de materiales mecánicos, pero sí sufre obsolescencia conceptual ante el cambio de los requerimientos.
2. **Costo marginal de duplicación nulo:** Copiar un componente software infinitas veces cuesta cero céntimos; levantar un segundo rascacielos idéntico requiere el mismo cemento, acero y mano de obra que el primero.

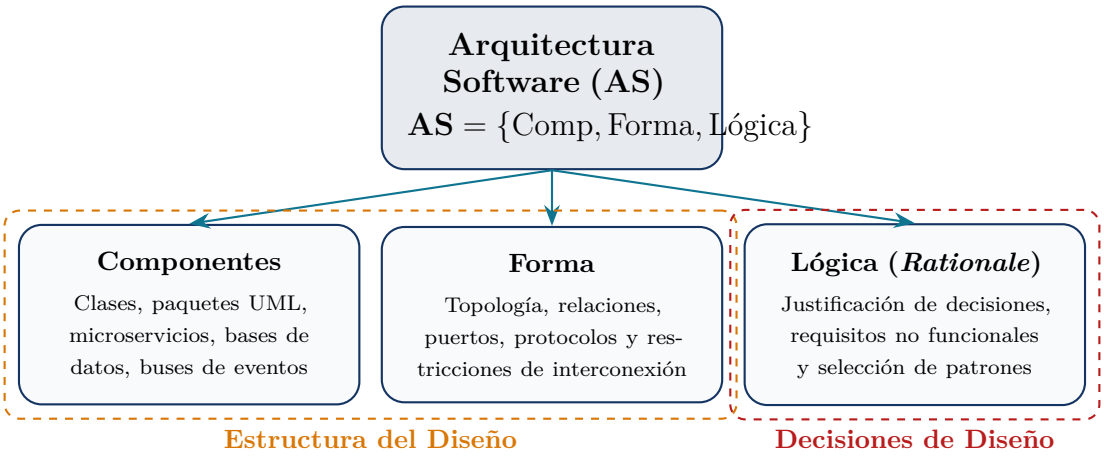



Figura 1.1: Desglose de la definición de Perry & Wolf: Estructura del diseño y decisiones de diseño.

| Elemento Civil | Equivalente en Software | Función y Rol Técnico |
|---|---|---|
| **Ladrillos** | Módulos y funciones elementales | Unidades mínimas de código ejecutable que resuelven operaciones atómicas. |
| **Bloques prefabr.** | Componentes software reutilizables | Librerías de terceros (npm, Maven, NuGet), frameworks empaquetados, SDKs. |
| **Cemento** | Conectores de software | Mecanismos de comunicación: llamadas de procedimiento, middleware, colas AMQP, gRPC, sockets. |
| **Moldes / Planos** | Patrones de diseño y estilos | Estructuras probadas de antemano (e.g., Singleton, Observer, Modelo-Vista-Controlador, Microservicios). |
| **Normas edificación** | Estándares, linters y contratos de interfaz | Restricciones normativas de calidad, seguridad y buenas prácticas impuestas al equipo. |

Tabla 1.1: Correspondencia entre arquitectura civil y arquitectura de software.

3. **Tasa de modificación continua (*Malleability*):** En una obra civil, es impensable ensanchar los cimientos una vez levantada una torre de sesenta pisos. En software, el negocio exige modificar el núcleo funcional en cada ciclo semanal de entrega (*Agile/CI/CD*). Por ello, la arquitectura software debe diseñarse explícitamente para admitir la **evolución continua** y el cambio controlado.

### 1.1.5 El Rol del Arquitecto Software en la Organización

Antiguamente, el arquitecto de sistemas solía concebirse como el "arquitecto de torre de marfil" (*Ivory Tower Architect*), un profesional aislado que producía densos volúmenes de documentación UML y ordenaba a los programadores implementarla sin intervenir en el código. Hoy en día, dicho rol está totalmente superado.

El arquitecto software moderno es un **líder técnico estratégico** cuya misión principal es actuar de puente entre dos universos:

- **El espacio del problema (Negocio):** Requisitos funcionales (lo que el sistema debe hacer comercialmente) y restricciones de tiempo y presupuesto.
- **El espacio de la solución (Ingeniería):** Requisitos no funcionales o atributos de calidad (disponibilidad del 99.999 %, latencia máxima de 15 ms, cumplimiento de estándares de cifrado militar).

El arquitecto software no toma decisiones para satisfacer gustos personales o tendencias pasajeras del mercado, sino para **resolver compromisos técnicos (*trade-offs*)**. Si un sistema requiere altísima velocidad de escritura a escala global, el arquitecto sabrá que según el teorema CAP deberá relajar la consistencia inmediata en favor de la consistencia eventual. La arquitectura software es, ante todo, la gestión consciente y fundamentada de los trade-offs.

### 1.1.6 Desalineación (*Mismatch*) y Erosión Arquitectónica

Durante el ciclo de vida de un sistema, la arquitectura prevista rara vez se mantiene inmutable. Dos fenómenos de vital importancia en la teoría y práctica de la ingeniería son:

**▶ Concepto Clave: Architectural Mismatch (Desalineación Arquitectónica)**

Descrito originalmente por David Garlan, Robert Allen y John Ockerbloom en 1995, el **architectural mismatch** ocurre al intentar integrar dos o más componentes software independientes que fueron desarrollados bajo **asunciones implícitas incompatibles**.

Estas asunciones incompatibles suelen clasificarse en:

- *Asunciones sobre el control:* ¿Qué componente alberga el bucle principal de ejecución? (ej., dos bibliotecas que ambas asumen ser las propietarias exclusivas del hilo de la interfaz gráfica).
- *Asunciones sobre los datos:* Formato, semántica y propiedad del estado de los datos transferidos (ej., representación binaria big-endian vs. little-endian, formatos JSON enriquecidos vs. arrays planos).
- *Asunciones sobre el entorno y protocolos:* Dependencias de hilos, versiones de runtime, modelos síncronos versus asíncronos.

**▶ Concepto Clave: Architectural Drift y Architectural Erosion (Erosión Arquitectónica)**

- **Architectural Drift (Deriva Arquitectónica):** Introducción paulatina de modificaciones o extensiones en el código que no fueron contempladas en la arquitectura original pero que no violan directamente sus reglas fundamentales.
- **Architectural Erosion (Erosión / Degradación Arquitectónica):** Violación flagrante y continuada de las reglas, fronteras y directrices arquitectónicas originales (por ejemplo, llamadas directas desde la capa de vista a la base de datos para "ahorrar tiempo" en una

entrega urgente).

La consecuencia inevitable de la erosión es la acumulación masiva de **deuda técnica arquitectónica**: el sistema se vuelve frágil, impredecible, cualquier cambio en un módulo desencadena fallos inesperados en subsistemas lejanos (*ripple effect*) y el coste de añadir nuevas características crece de forma exponencial hasta forzar la reescritura completa del sistema.

**✓ Ejemplo Práctico: Detección automatizada de la erosión con Architecture-as-Code**

En la ingeniería moderna, para evitar que la arquitectura de un sistema degrade con el paso de los sprints se emplean **pruebas unitarias de arquitectura** (con herramientas como *ArchUnit* en Java/.NET o linters de dependencias modulares). Un test de arquitectura se define en código:

```
@Test
public void capas_deben_respetar_direccion_estricta() {
    noClasses()
        .that().resideInAPackage("..domain..")
        .should().dependOnClassesThat()
        .resideInAPackage("..infrastructure..")
        .because("Las entidades de dominio nunca deben depender de detalles de base de datos o frameworks externos (DIP)")
        .check(classes);
}
```

Listing 1.1: Ejemplo de test con ArchUnit para impedir la erosión arquitectónica

Si algún desarrollador intenta saltarse la arquitectura inyectando un driver JDBC en una entidad del dominio, el pipeline de Integración Continua (CI) falla de inmediato.

## 1.2 Modelos de Referencia, Arquitecturas de Referencia y Familias de Productos

Uno de los temas más evaluados y de mayor impacto estratégico en empresas que desarrollan software a gran escala es el paso desde la solución aislada a la reutilización sistemática.

### 1.2.1 Distinción entre Modelo de Referencia y Arquitectura de Referencia

Existe una jerarquía formal en el nivel de abstracción que conecta un problema genérico con una implementación concreta:

**▶ Concepto Clave: Modelo de Referencia**

Un **Modelo de Referencia** es una división conceptual de la funcionalidad requerida por un dominio de problemas, estructurada en bloques funcionales con flujos de datos e información claramente definidos entre ellos, pero **completamente desacoplada de cualquier decisión de implementación software específica**.

**✓ Ejemplo Práctico: Ejemplos clásicos de Modelos de Referencia**

- **Compilador clásico:** Cualquier compilador se descompone conceptualmente en: *Analizador Léxico* → *Analizador Sintáctico* → *Analizador Semántico* → *Generador de Código Intermedio* → *Optimizador* → *Generador de Código Destino*. Esta descomposición es un modelo de referencia; no especifica si el compilador se escribe en C++, Rust o Java, ni si se compila en un único ejecutable o en herramientas separadas conectadas por tuberías.

- **Modelo OSI de 7 Capas de ISO:** Define abstractamente qué debe hacer la capa de transporte, de red, de enlace, etc., sin imponer sockets POSIX, tarjetas ethernet específicas ni protocolos propietarios.

**▶ Concepto Clave: Arquitectura de Referencia**

Una **Arquitectura de Referencia** es el resultado de mapear o proyectar un **Modelo de Referencia** sobre componentes de software reales, protocolos concretos, flujos de datos operacionales y estilos arquitectónicos preestablecidos.

Una arquitectura de referencia no es todavía el producto final comercial, sino una **plantilla arquitectónica probada y validada** que sirve de guía para construir múltiples productos dentro de un mismo dominio vertical.

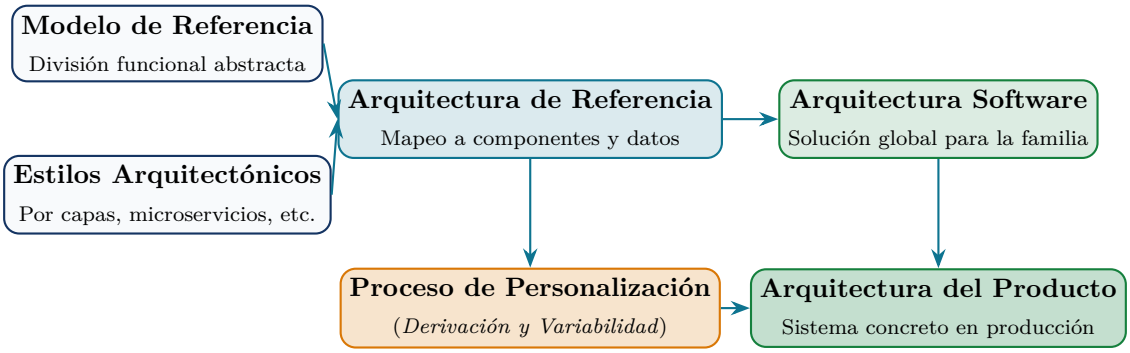



Figura 1.2: Flujo conceptual: De los modelos y estilos a la arquitectura del producto software final.

### 1.2.2 Caso de Estudio Industrial: AUTOSAR en la Automoción

Uno de los ejemplos de mayor relevancia en sistemas embebidos de tiempo real y misión crítica es el estándar **AUTOSAR** (*Automotive Open System Architecture*).

Antes de AUTOSAR, cada fabricante de automóviles (OEM) y proveedor de primer nivel (Tier 1) desarrollaba el software de las centralitas electrónicas (ECUs - *Engine Control Units*) mezclando la lógica de control del motor o frenos con los drivers de bajo nivel del microcontrolador específico. Si el fabricante cambiaba de chip de silicio, el 100 % del software debía desecharse y reescribirse.

**▶ Concepto Clave: Estructura de la Arquitectura de Referencia AUTOSAR**

- **Application Layer:** Contiene los componentes funcionales (*SW-Cs*), tales como el algoritmo de control de crucero adaptativo o la gestión del airbag. Estos componentes ignoran por completo qué microcontrolador corre por debajo o por qué bus físico viajan los mensajes.
- **AUTOSAR Runtime Environment (RTE):** Es el middleware central. Actúa como un conector universal (*Virtual Functional Bus*). Cualquier llamada entre componentes se realiza a través de las interfaces estándar del RTE, garantizando portabilidad absoluta.
- **Basic Software (BSW):** Provee los servicios esenciales agrupados en:
  - *Servicios de Sistema y SO:* Planificador de tareas en tiempo real (OSEK/VDX modificado).
  - *Pila de Comunicaciones:* Abstracción de protocolos de automoción (CAN, LIN, FlexRay, Automotive Ethernet).
  - *Capa de Abstracción de ECU:* Aisla las peculiaridades de la placa de circuito impreso.
  - *MCAL (Microcontroller Abstraction Layer):* Es la única capa dependiente del silicio que mapea los registros del microcontrolador.

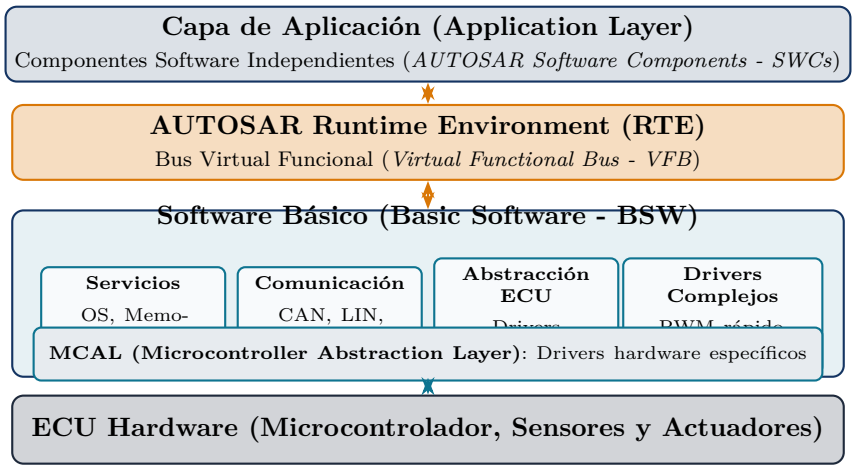



Figura 1.3: Arquitectura de Referencia AUTOSAR: Desacoplamiento radical mediante el RTE.

### 1.2.3 De Sistemas Aislados a Familias de Sistemas (*Software Product Lines*)

Históricamente, las organizaciones de software operaban bajo el paradigma del **diseño artesanal por producto**: para cada nuevo cliente o variante de software se creaba un proyecto independiente bifurcando el código anterior (*copy-paste-modify*). Este enfoque resulta catastrófico: los parches de seguridad deben aplicarse manualmente $N$ veces y los costes de mantenimiento crecen a un ritmo inasumible.

El enfoque moderno se fundamenta en las **Familias de Sistemas** o **Líneas de Producto Software (Software Product Lines - SPL)**:

> ▶ **Concepto Clave: Líneas de Producto Software (SPL)**
>
> Una Línea de Producto Software es un paradigma de ingeniería de software para generar colecciones de sistemas similares a partir de un conjunto compartido de activos software base (*core assets*), explotando sistemáticamente sus **puntos comunes** (*commonalities*) y gestionando formalmente sus **puntos de variación** (*variabilities*).

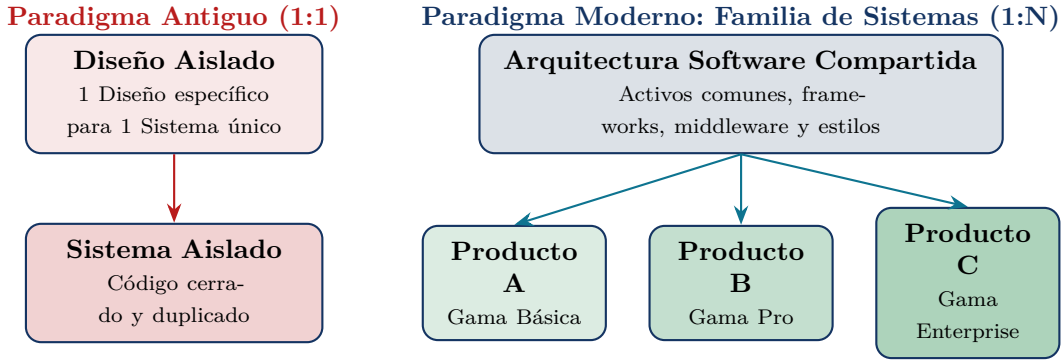



Figura 1.4: Transición desde el diseño artesanal aislado a la arquitectura de familias de productos.

### 1.2.4 El Proceso de Personalización y la Gestión de la Variabilidad

La "magia" de una buena arquitectura software radica en que un porcentaje altísimo del sistema (frecuentemente entre el 70 % y el 90 %) reside en las **partes comunes** (*core assets*): autenticación, transaccionalidad, persistencia, telemetría y comunicación.

Las partes variables representan los aspectos que diferencian a un producto concreto de otro. La **variabilidad software** puede resolverse en distintos instantes del ciclo de vida:

- **En tiempo de diseño / desarrollo:** Mediante herencia, polimorfismo, interfaces y aplicación del patrón *Strategy* o *Template Method*.
- **En tiempo de compilación / empaquetado:** Mediante preprocesadores, compilación condicional (e.g., directivas en C/C++ o Rust) o empaquetado modular (*tree-shaking*, perfiles de Maven/Gradle).
- **En tiempo de despliegue / arranque:** Mediante ficheros de configuración, variables de entorno e inyección de dependencias parametrizada en el contenedor IoC.
- **En tiempo de ejecución (*Runtime*):** Mediante arquitectura de plugins dinámicos (*reflection*), carga perezosa (*lazy loading*) o banderas de características en vivo (*Feature Flags / Toggles*).

## 1.3 Principios Fundamentales de Diseño Software

La arquitectura de un sistema no es producto del azar ni de la inspiración artística; se sustenta sobre un conjunto inmutable de **principios de diseño** formulados por los pioneros de la computación. Violar estos principios degrada irreversiblemente cualquier arquitectura, con independencia de los frameworks o tecnologías utilizados.

### 1.3.1 Separación de Responsabilidades (*Separation of Concerns - SoC*)

Formulado originariamente por Edsger W. Dijkstra en su ensayo de 1974 *"On the role of scientific thought"*, el principio de Separación de Responsabilidades establece que:

> ▶ **Concepto Clave: Separation of Concerns (SoC)**
>
> Un sistema complejo debe descomponerse en secciones o módulos diferenciados de forma que cada uno de ellos se encargue de una preocupación o responsabilidad única y claramente delimitada, minimizando de raíz los solapamientos de funcionalidad.

SoC opera a todos los niveles de granularidad arquitectónica:

- **Nivel de Capas (Layers):** Separar rigurosamente la *Capa de Presentación* (UI / API REST), la *Capa de Negocio o Dominio* (cálculos, validaciones, reglas comerciales) y la *Capa de Persistencia e Infraestructura* (bases de datos, colas de mensajería, llamadas a APIs externas).
- **Nivel de Paquetes UML / Módulos:** Agrupar elementos por coherencia funcional (e.g., módulo de facturación, módulo de envíos, módulo de usuarios).
- **Nivel de Clases:** Cumplir el principio de Responsabilidad Única (SRP - *Single Responsibility Principle*): cada clase debe tener un único motivo para cambiar.

### 1.3.2 Acoplamiento y Cohesión: El Binomio Maestro

La calidad estructural de cualquier arquitectura software se mide mediante dos propiedades contrapuestas y complementarias: **Acoplamiento** y **Cohesión**.

> ▶ **Concepto Clave: Acoplamiento (Coupling)**
>
> El **acoplamiento** mide el **grado de interdependencia entre dos o más módulos o bloques** de una arquitectura. Indica con cuánta fuerza depende un componente de los detalles internos de otro.
> **Objetivo Arquitectónico:** Conseguir **Bajo Acoplamiento (*Loose Coupling*)**.

▶ **Concepto Clave: Cohesión (Cohesion)**

La **cohesión** mide la **fuerza y relevancia con la que los elementos internos (variables, métodos, submódulos) dentro de un mismo componente trabajan juntos** para cumplir una única función bien definida.
**Objetivo Arquitectónico:** Conseguir **Alta Cohesión (*High Cohesion*)**.

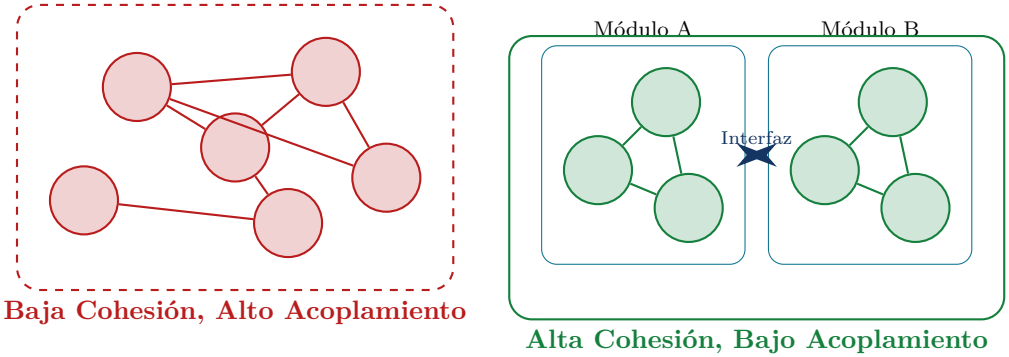



Figura 1.5: Comparativa estructural: sistemas caóticos acoplados frente a sistemas modulares altamente cohesivos y desacoplados.

Para responder con matrícula de honor en un examen técnico o desenvolverse en una empresa de primer nivel, el ingeniero debe dominar la clasificación de acoplamiento y cohesión propuesta por Stevens, Myers y Constantine:

| Nivel de Acoplamiento | Naturaleza de la Interacción | Veredicto de Ingeniería |
|---|---|---|
| **De Contenido (*Content*)** | Un módulo modifica directamente datos o código interno de otro módulo. | **Inaceptable**. Rompe cualquier encapsulamiento. |
| **Común / Global (*Common*)** | Múltiples módulos comparten y mutan las mismas variables o memorias globales. | **Peligroso**. Causa efectos colaterales impredecibles y condiciones de carrera. |
| **De Control (*Control*)** | Un módulo pasa flags booleanos o códigos de operación para mandar qué camino ejecutar dentro de otro. | **Indeseable**. Conoce la lógica interna del receptor. |
| **De Estampado (*Stamp*)** | Se pasa una estructura de datos completa (ej. objeto Usuario con 40 campos) cuando la función sólo necesita 2 campos. | **Aceptable pero mejorable**. Acopla al esquema de datos completo. |
| **De Datos (*Data*)** | Los módulos se comunican pasando exclusivamente argumentos primitivos o contratos estrictos. | **Excelente**. Bajo acoplamiento. |
| **Por Mensajes (*Message*)** | Comunicación asíncrona mediante paso de mensajes o eventos sin conocimiento del receptor. | **Óptimo**. Máximo desacoplamiento espacial y temporal. |

Tabla 1.2: Espectro jerárquico de tipos de acoplamiento (de peor a mejor).

| Nivel de Cohesión | Principio de Agrupación | Veredicto de Ingeniería |
|---|---|---|
| **Coincidente** | Elementos reunidos al azar o por conveniencia sin relación real (e.g., típica clase `GeneralUtils`). | **Pésima**. Imposible de entender o mantener. |
| **Lógica** | Agrupa funciones de la misma categoría abstracta (e.g., una clase que procesa todas las entradas de ratón, teclado y red en un switch). | **Baja**. Viola SRP y mezcla contextos. |
| **Temporal** | Elementos que deben ejecutarse en el mismo instante temporal (e.g., inicializar base de datos, cargar fuentes y abrir ventana en un único método). | **Poco deseable**. Deberían delegar en sus respectivos subsistemas. |
| **Procedimental / Comunicac.** | Elementos que operan sobre los mismos datos de entrada/salida o siguen un flujo de pasos. | **Aceptable**. |
| **Secuencial** | La salida de una operación es la entrada directa e indispensable de la siguiente (estilo pipeline/tubería). | **Buena**. |
| **Funcional** | **Todos los elementos del componente cooperan para cumplir una única tarea bien delimitada** (ej., cálculo de ruta óptima en un grafo). | **Ideal**. Máxima cohesión posible. |

Tabla 1.3: Espectro jerárquico de tipos de cohesión (de peor a mejor).

### 1.3.3 Anti-patrones de Diseño Estructural: El Anti-patrón God Class

Cuando se ignoran la separación de responsabilidades y la alta cohesión, surge uno de los anti-patrones más destructivos del desarrollo orientado a objetos:

> ▶ **Concepto Clave: Anti-patrón God Class (The Blob)**
>
> Una **God Class** (o clase todoterreno) es una clase que concentra una cantidad desproporcionada de responsabilidades, estado y lógica de negocio de un sistema, relegando al resto de clases a simples contenedores pasivos de datos (*anemic data holders*).

**Patología y consecuencias de la God Class:**

- **Alta Complejidad Ciclomática:** Métodos interminables con decenas de niveles de anidamiento condicional (`if/switch`) para gestionar casos especiales.
- **Baja Cohesión:** Los métodos manipulan subconjuntos disjuntos de atributos de la clase; el índice métrico LCOM (*Lack of Cohesion in Methods*) se dispara.
- **Imposibilidad de realizar pruebas unitarias fiables:** Para testear una sencilla regla de cálculo se requiere instanciar una clase que arrastra conexiones de base de datos, clientes de correo y pasarelas de pago.
- **Bloqueo en el trabajo en equipo:** Múltiples desarrolladores colisionan constantemente en Git al modificar el mismo fichero masivo.

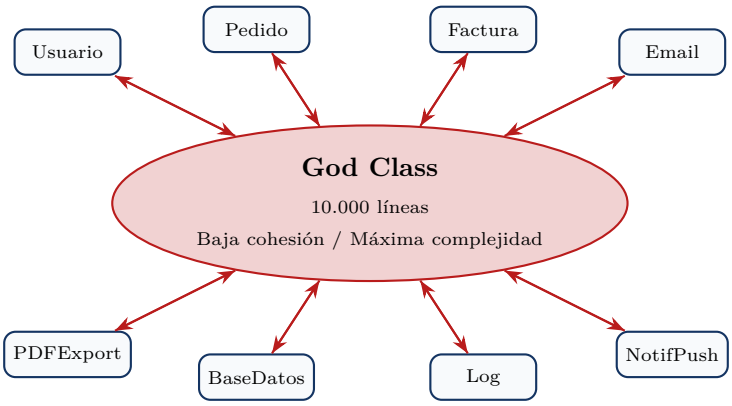



Figura 1.6: Topología estelar tóxica generada por una God Class.

### 1.3.4 Ocultamiento de Información (*Information Hiding*) y Encapsulamiento

Propuesto por David Parnas en su célebre artículo de 1972 (*"On the Criteria To Be Used in Decomposing Systems into Modules"*):

**► Concepto Clave: Ocultamiento de Información (Information Hiding)**

El principio de ocultamiento de información establece que cada módulo o componente debe **ocultar a todos los demás un secreto de diseño** (un algoritmo volátil, la estructura interna de una tabla de base de datos o el formato de serialización), exponiendo hacia el exterior únicamente una interfaz pública abstracta, estable y mínima.

**▲ ¡Atención / Ingeniería!: Diferencia técnica: Encapsulamiento vs. Information Hiding**

A menudo se usan como sinónimos, pero no lo son:

- **Encapsulamiento** es el mecanismo del lenguaje de programación que agrupa datos y las operaciones que actúan sobre ellos en una misma unidad (clase o struct), controlando la visibilidad mediante modificadores (`private`, `protected`, `public`).
- **Information Hiding** es un **principio de diseño arquitectónico**. Se puede tener una clase perfectamente encapsulada con getters y setters privados que sin embargo viole el Information Hiding si expone hacia el exterior detalles como la clave primaria autoincremental de la base de datos o si obliga al cliente a conocer el orden exacto de llamadas internas.

### 1.3.5 El Peligro de las Dependencias Circulares y el Principio ADP

Una dependencia circular ocurre cuando un conjunto de componentes depende directa o indirectamente de sí mismo: el módulo $A$ depende de $B$, $B$ depende de $C$, y $C$ depende de nuevo de $A$.

**¿Por qué las dependencias circulares son destructivas?**

1. **Fusión forzada de módulos:** Aunque estén en paquetes o repositorios distintos, los componentes acoplados circularmente ya no son módulos independientes; forman un único bloque monolítico indivisible.
2. **Efecto Onda (*Ripple Effect*):** Cualquier modificación en una interfaz de $A$ obliga a recomprobar y recompilar $B$ y $C$, generando un ciclo de retroalimentación infinita de cambios.
3. **Paradojas de Inicialización:** En tiempo de ejecución, el contenedor o el runtime no sabe qué objeto inicializar primero, provocando bloqueos mutuos (*deadlocks*) o referencias a memoria nula (*NullPointerExceptions*).




Figura 1.7: Dependencia circular entre tres subsistemas.

4. **Imposibilidad de pruebas unitarias aisladas:** No se puede probar $A$ sin arrastrar el entorno entero de $B$ y $C$.

> ► **Concepto Clave: Principio de Dependencias Acíclicas (Acyclic Dependencies Principle - ADP)**
>
> El grafo de dependencias entre componentes, módulos o paquetes de un sistema debe ser estrictamente un **Grafo Acíclico Dirigido (DAG)**. No se permiten ciclos en la estructura de dependencias de la arquitectura.

**¿Cómo romper un ciclo de dependencias circular?** Existen dos estrategias fundamentales de ingeniería:

- **Estrategia 1: Inversión de Dependencias (DIP):** Si $C$ depende de $A$, introducimos una interfaz $I$ en el módulo $C$ que define el comportamiento requerido. Hacemos que $A$ implemente la interfaz $I$. De esta forma, la flecha de dependencia se invierte y el ciclo se destruye.
- **Estrategia 2: Extracción a un Módulo Común:** Si $A$ y $B$ dependen mutuamente porque ambos usan un tipo de dato o entidad compartida, se extrae esa entidad a un nuevo módulo inferior $D$ del que ambos dependen en un único sentido ($A \to D$ y $B \to D$).

## 1.4 Inversión de Dependencias (DIP), Inversión de Control (IoC) e Inyección de Dependencias (DI)

Pocas áreas causan tanta confusión entre profesionales y estudiantes como las siglas **DIP**, **IoC** y **DI**. Aclararemos de forma incontrovertible qué significa cada término, a qué nivel de la ingeniería aplica y cómo se relacionan entre sí.

> ► **Concepto Clave: Desambiguación Conceptual Definitiva**
>
> - **Inversion of Control (IoC):** Es un **principio arquitectónico general**. Se refiere al cambio en la dirección habitual del flujo de control en un sistema: el marco de trabajo (*framework*) toma el control y orquesta la ejecución llamando al código de la aplicación.
> - **Dependency Inversion Principle (DIP):** Es un **principio de diseño de software orientado a objetos** (la 'D' de los principios SOLID). Dictamina que los módulos de negocio de alto nivel no deben depender de los detalles técnicos de bajo nivel, sino de abstracciones.
> - **Dependency Injection (DI):** Es un **patrón de diseño de software y técnica de implementación**. Es el mecanismo concreto mediante el cual un objeto recibe sus dependencias externas desde una entidad inyectora en lugar de instanciarlas él mismo. Es la forma más habitual de materializar DIP e IoC.

### 1.4.1 El Principio de Inversión de Dependencias (DIP)

Enunciado por Robert C. Martin (“Uncle Bob”) en 1996, el principio establece dos leyes formales:

▶ **Concepto Clave: Leyes Formales de DIP**

1. **Los módulos de alto nivel no deben depender de los módulos de bajo nivel. Ambos deben depender de abstracciones (interfaces o clases abstractas).**
2. **Las abstracciones no deben depender de los detalles. Los detalles (las implementaciones concretas) deben depender de las abstracciones.**

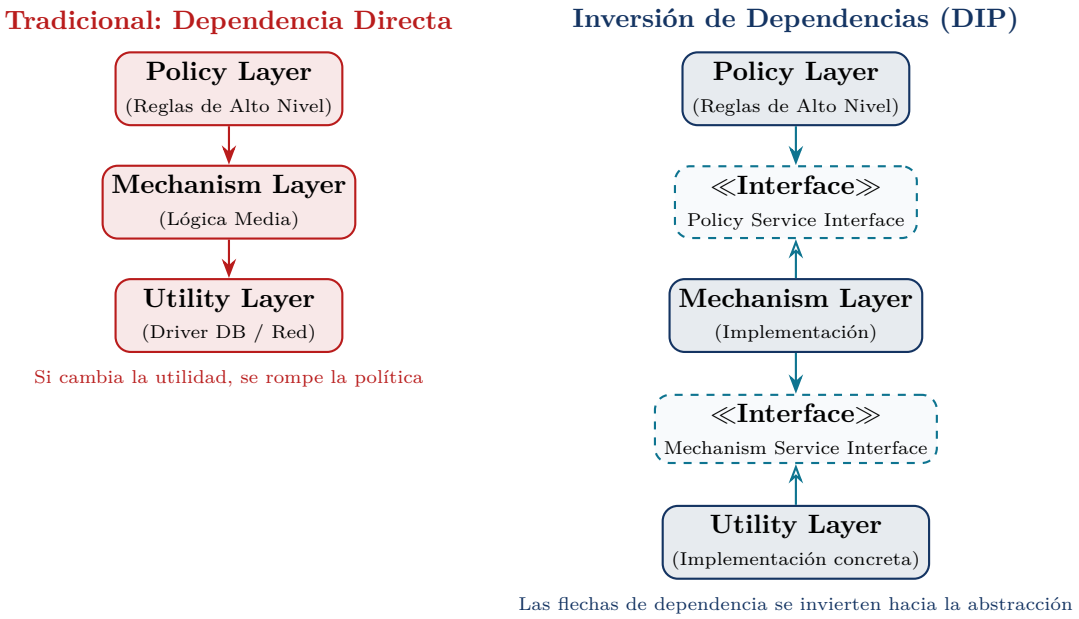



Figura 1.8: Comparativa: flujo de dependencias sin DIP frente a inversión mediante interfaces.

▲ **¡Atención / Ingeniería!: ¿Por qué se llama “Inversión”?**

En el diseño tradicional procedural, el flujo de control y las dependencias de código fuente viajan en la misma dirección: la función principal llama a la subrutina y el archivo fuente incluye directamente el archivo de la subrutina. Con DIP, **el flujo de dependencias de código fuente viaja en sentido contrario al flujo de control**:

- En tiempo de ejecución (*Control Flow*): La capa de negocio invoca la operación y el código de bajo nivel la ejecuta.
- En el código fuente (*Dependency Flow*): El módulo de bajo nivel importa y depende de la interfaz definida por el módulo de alto nivel.

### 1.4.2 Caso Práctico de Código: Violación y Corrección de DIP

Analicemos un caso habitual de una aplicación web: un servicio de autenticación (`AuthService`) que autentica a un usuario y le envía un correo electrónico.

#### Caso Incorrecto: Violación flagrante de DIP

```
1 // Clase de infraestructura concreta (Bajo nivel)
2 class MongoDBUserRepository {
```

```
  getUser(email) {
    // Consulta directa a coleccion nativa de MongoDB
    return db.collection("users").findOne({ email: email });
  }
}

// Clase de infraestructura concreta (Bajo nivel)
class GmailMailerService {
  sendLoginEmail(email) {
    // Llamada directa a la API HTTP propietaria de Google Gmail
    console.log("Enviando correo mediante API propietaria de Gmail a " + email);
  }
}

// Modulo de Negocio (Alto nivel)
class AuthenticationService {
  constructor() {
    // ERROR ARQUITECTONICO: La clase de alto nivel crea y se acopla rígidamente
    // a implementaciones concretas de bajo nivel.
    this.repository = new MongoDBUserRepository();
    this.mailer = new GmailMailerService();
  }

  login(email) {
    const user = this.repository.getUser(email);
    if (user) {
      this.mailer.sendLoginEmail(email);
      return true;
    }
    return false;
  }
}
```

Listing 1.2: Código acoplado directamente a implementaciones concretas (Sin DIP)

**Problemas graves de este diseño:**

1. Si mañana la empresa decide migrar de MongoDB a PostgreSQL o DynamoDB, hay que modificar el código fuente de `AuthenticationService`.
2. Si Gmail cambia sus credenciales por SendGrid o Amazon SES, hay que reescribir `AuthenticationService`.
3. **Es imposible testear en aislamiento:** No se puede ejecutar una prueba unitaria rápida de `login()` sin levantar una base de datos MongoDB real y sin mandar correos electrónicos auténticos por Gmail.

### Caso Correcto: Aplicando DIP con Interfaces y Desacoplamiento

Refactoricemos introduciendo abstracciones:

```
// 1. DEFINICION DE ABSTRACCIONES (CONTRATOS)
// En TypeScript / Java / C# serian interfaces estrictas:
// interface UserRepository { findByEmail(email: string): User | null; }
// interface MailService { sendEmail(to: string, message: string): void; }

// 2. MODULO DE ALTO NIVEL (REGLAS DE NEGOCIO PURAS)
class AuthenticationService {
  // El modulo de negocio depende unicamente de abstracciones.
  // No sabe si los datos vienen de Mongo, Postgres, memoria o un archivo.
  constructor(userRepository, mailService) {
    this.userRepository = userRepository;
    this.mailService = mailService;
  }

  login(email) {
    const user = this.userRepository.findByEmail(email);
    if (user) {
```

```
18        this.mailService.sendEmail(email, "Ha iniciado sesion con exito");
19        return true;
20      }
21      return false;
22    }
23  }
24
25  // 3. IMPLEMENTACIONES CONCRETAS DE BAJO NIVEL (ADAPTADORES)
26  class MongoDBUserRepository /* implements UserRepository */ {
27    findByEmail(email) {
28      // Logica especifica de MongoDB
29      return db.collection("users").findOne({ email });
30    }
31  }
32
33  class SendGridMailService /* implements MailService */ {
34    sendEmail(to, message) {
35      // Integracion con la API de SendGrid
36      console.log(`[SendGrid] Mensaje enviado a ${to}: ${message}`);
37    }
38  }
```

Listing 1.3: Código desacoplado que cumple rigurosamente con DIP

### 1.4.3 Inversión de Control (IoC) y el Principio de Hollywood

**▶ Concepto Clave: Principio de Hollywood**

*"Don't call us, we'll call you"* (No nos llames tú a nosotros; nosotros te llamaremos a ti).

En la programación tradicional con librerías, el código del desarrollador es el que tiene la iniciativa: el programa arranca en la función `main()`, procesa datos y llama a los métodos de la librería cuando los necesita. La librería es un actor pasivo.

En la **Inversión de Control**, se invierte la iniciativa: el framework o el contenedor es el dueño del bucle de ejecución principal. El desarrollador escribe pequeñas piezas de lógica (controladores, manejadores de eventos, plugins) y las registra en el framework. Es el framework el que, ante un estímulo externo (un clic del ratón, un paquete de red o una petición HTTP entrante), decide cuándo invocar al código del desarrollador.

**✓ Ejemplo Práctico: La Inversión de Control en Sistemas Operativos y GUIs**

Pensemos en el sistema de ventanas de Windows o en el DOM del navegador:

- En lugar de que el programa de un estudiante ejecute un bucle infinito consultando activamente: `while(true) { if(raton_pulsado()) ... }` (sondeo activo o *polling*, que saturaría la CPU al 100 %).
- El sistema operativo Windows gestiona el bucle de mensajes del sistema (*Message Loop*). Nosotros únicamente implementamos un manejador (*event handler*) de `OnClick`. Cuando el usuario mueve el ratón o pulsa un botón, Windows intercepta la interrupción hardware y es el sistema operativo quien invoca nuestro manejador de eventos. ¡Eso es pura Inversión de Control!

### 1.4.4 Inyección de Dependencias (DI): Mecanismos y Roles

La **Inyección de Dependencias** es el patrón técnico que materializa IoC para la provisión de dependencias colaboradoras entre objetos.

En lugar de que un componente $A$ construya internamente un objeto $B$ con el operador `new`, un agente externo especializado (**el Inyector**) crea $B$ y se lo "entrega" a $A$.

En cualquier diseño con DI intervienen de forma nítida tres roles:

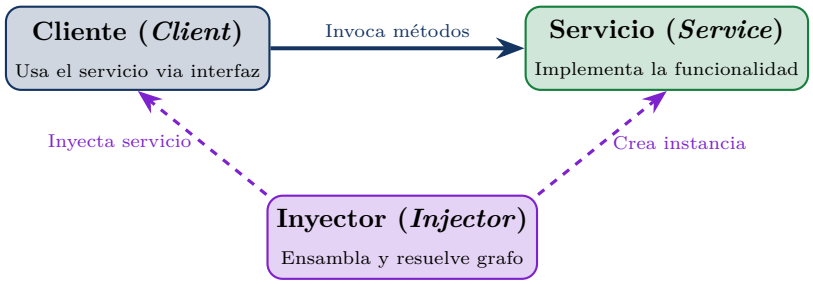



Figura 1.9: La tríada de Dependency Injection: Cliente, Servicio e Inyector.

1. **Cliente (*Client*):** La clase que requiere uno o más servicios para realizar su cometido (ej., `AuthenticationService`). El cliente no conoce la clase concreta del servicio; solo conoce su contrato/interfaz.
2. **Servicio (*Service*):** La clase concreta que implementa el contrato (ej., `MongoDBUserRepository`).
3. **Inyector (*Injector / IoC Container*):** El componente (como el contenedor de Spring, NestJS o un inicializador manual) responsable de instanciar el servicio y pasárselo al cliente.

### 1.4.5 Modalidades de Inyección de Dependencias

Existen tres formas estándar en las que el inyector puede entregar la dependencia al cliente:

1. **Inyección por Constructor (*Constructor Injection*):**
   Las dependencias se declaran como parámetros obligatorios del constructor de la clase.

```
class OrderService {
  constructor(paymentGateway, inventoryService) {
    this.paymentGateway = paymentGateway;
    this.inventoryService = inventoryService;
  }
}

```

   **Evaluación:** Es la **opción unánimemente recomendada por la industria**, ya que garantiza que el objeto nunca pueda existir en un estado inconsistente o parcialmente inicializado, fomenta la inmutabilidad (`final / readonly`) y hace inmediatamente visibles las dependencias de la clase.

2. **Inyección por Métodos Modificadores (*Setter / Property Injection*):**
   Las dependencias se suministran a través de métodos setter después de que el objeto ha sido construido.

```
class OrderService {
  setPaymentGateway(paymentGateway) {
    this.paymentGateway = paymentGateway;
  }
}

```

   **Evaluación:** Solo debe emplearse para dependencias verdaderamente **opcionales** o valores de configuración por defecto que pueden reconfigurarse en caliente. Su riesgo es que la clase puede llamarse antes de invocar al setter, arrojando excepciones en tiempo de ejecución.

3. **Inyección por Parámetro de Método (*Method / Interface Injection*):**
   La dependencia no se almacena como atributo de la clase, sino que se suministra únicamente como parámetro al método particular que la necesita para una operación puntual.

### 1.4.6 Impacto Decisivo en la Testeabilidad: Dobles de Prueba

El mayor beneficio de ingeniería que aporta la combinación DIP + DI no es estético: es la **testeabilidad radical del sistema**.

Cuando un componente crea sus propias dependencias con `new`, es imposible probarlo sin conectarse a la infraestructura real. Cuando las dependencias se inyectan a través de interfaces, en las pruebas unitarias podemos sustituir las dependencias reales pesadas por **Dobles de Prueba (*Test Doubles*)**:

```
// En un fichero de test unitario (sin tocar base de datos ni red):
describe("AuthenticationService Test Suite", () => {
  it("debe retornar true y enviar email cuando las credenciales son validas", () => {
    // 1. Creamos un Mock/Stub en memoria del repositorio
    const mockRepo = {
      findByEmail: (email) => ({ id: 42, email: "test@urjc.es" })
    };

    // 2. Creamos un Mock espía del servicio de correo
    let emailEnviado = false;
    const mockMailer = {
      sendEmail: (to, msg) => { emailEnviado = true; }
    };

    // 3. Inyectamos los dobles de prueba en el servicio a testear
    const authService = new AuthenticationService(mockRepo, mockMailer);

    // 4. Ejecutamos la logica de negocio
    const result = authService.login("test@urjc.es");

    // 5. Aseveraciones matematicamente deterministicas en < 1 milisegundo
    expect(result).toBe(true);
    expect(emailEnviado).toBe(true);
  });
});
```

Listing 1.4: Prueba Unitaria ultrarrápida en memoria gracias a DIP y DI

## 1.5 Ejercicios Resueltos y Casos Prácticos de Ingeniería

Para afianzar de forma definitiva los conocimientos y garantizar la máxima calificación en exámenes teóricos y pruebas de diseño técnico en empresas punteras, analizaremos tres casos prácticos exhaustivos.

**★ Ejercicio Resuelto: Formalización de Arquitectura según la Fórmula de Perry & Wolf**

**Enunciado:**
Una empresa de telemedicina desea diseñar una plataforma en la nube para monitorizar signos vitales de pacientes ingresados en hospitales mediante pulseras IoT. Los sensores transmiten cada segundo datos de frecuencia cardíaca y saturación de oxígeno por protocolo MQTT. Un broker ingiere los mensajes, un servicio procesador detecta anomalías cardíacas y, si hay emergencia, dispara una alerta inmediata a una aplicación móvil de médicos de guardia vía notificaciones Push (WebSockets/APNs) y almacena el registro histórico en una base de datos Time-Series.
**Tarea del Ingeniero:**
Especifique formalmente la arquitectura software de este sistema de acuerdo con la definición canónica de Perry & Wolf: **AS** $= \{\text{Componentes}, \text{Forma}, \text{Lógica}\}$.

**Solución Razonada:**

1. **Componentes ($C$):**
   - *Elementos de Datos ($C_D$):* MedicionSignoVitalDTO, AlertaCriticaEvent, HistoricoPacienteRecord.
   - *Elementos de Procesamiento ($C_P$):* IngestionService, AnomalyDetectionEngine, AlertNotifi-

cationManager, TimeSeriesPersistenceWorker.
   - *Elementos de Conexión ($C_C$):* Broker MQTT (EMQX/Mosquitto), Bus de Eventos interno (Kafka/RabbitMQ), Pasarela Push (Apple APNs / Google FCM), Driver de base de datos TimeSeries (InfluxDB/Timescale).
2. **Forma ($F$):**
   - *Restricciones Topológicas:* Estilo Arquitectónico Orientado a Eventos (*Event-Driven*).
   - Los dispositivos IoT solo tienen visibilidad del puerto de entrada del Broker MQTT mediante canal cifrado TLS 1.3 con autenticación mutua (mTLS).
   - El *AnomalyDetectionEngine* se suscribe al canal de telemetría y procesa las mediciones en streaming. No accede a la base de datos histórica para evitar latencias de I/O.
   - Si se detecta una anomalía (HR $>$ 150 bpm o $SpO_2 < 90\,\%$), publica un evento *AlertaCriticaEvent* al bus de eventos. El *AlertNotificationManager* consume de dicho bus y abre canal directo WebSocket con el terminal del médico.
3. **Lógica / Rationale ($L$):**
   - *¿Por qué MQTT en lugar de HTTP REST?* Debido a la restricción de bajo consumo energético y escaso ancho de banda de las pulseras IoT embebidas; el protocolo binario ligero de MQTT minimiza la sobrecarga de cabeceras.
   - *¿Por qué una base de datos Time-Series?* Para optimizar la tasa de escritura masiva (miles de muestras por segundo) y permitir agregaciones y descarte automático de métricas antiguas (*retention policies*).
   - *¿Por qué desacoplamiento mediante Bus de Eventos?* Para garantizar que una caída temporal en el almacenamiento histórico no interrumpa ni retrase la entrega en tiempo real de alertas que salvan vidas (disponibilidad y tolerancia a fallos desacopladas).

### ★ Ejercicio Resuelto: Detección y Refactorización de Violación de DIP y Ciclo de Dependencias

**Enunciado:**
Se presenta el siguiente diagrama y fragmento de código de un sistema de comercio electrónico en el que participan tres subsistemas: `OrderService`, `InvoiceService` y `PdfGenerator`:

```
// En modulo de Facturas:
class InvoiceService {
  constructor() {
    this.pdf = new PdfGenerator();
  }
  generateInvoice(order) {
    // Genera factura para el pedido
    const doc = this.pdf.buildPdf(order);
    return doc;
  }
}

// En modulo de Generacion PDF:
class PdfGenerator {
  buildPdf(order) {
    // Para formatear, el generador de PDF vuelve a llamar a OrderService
    const orderService = new OrderService();
    const customerInfo = orderService.lookupCustomer(order.id);
    return "PDF_CONTENT with " + customerInfo;
  }
}

// En modulo de Pedidos:
class OrderService {
  constructor() {
    this.invoiceService = new InvoiceService();
  }
```

```
  checkout(order) {
    // Guarda el pedido y emite factura
    return this.invoiceService.generateInvoice(order);
  }
  lookupCustomer(orderId) {
    return "Cliente URJC";
  }
}
```

**Cuestiones a resolver:**

1. Identifique y justifique todas las violaciones a los principios de diseño presentes.
2. Dibuje el grafo de dependencias original demostrando el ciclo.
3. Proponga el diseño refactorizado aplicando DIP e Inyección de Dependencias, eliminando por completo el ciclo.

**Solución Razonada:**

**1. Violaciones identificadas:**

- **Violación flagrante del ADP (Acyclic Dependencies Principle):** Existe una dependencia circular cerrada:

$$\text{OrderService} \longrightarrow \text{InvoiceService} \longrightarrow \text{PdfGenerator} \longrightarrow \text{OrderService}$$

- **Violación del Principio de Inversión de Dependencias (DIP):** Cada clase instancia directamente a sus colaboradoras mediante `new`, acoplándose a implementaciones concretas en vez de abstracciones.
- **Violación de Separation of Concerns (SoC) e Information Hiding:** El componente de bajo nivel utilitario (`PdfGenerator`) conoce detalles del dominio de pedidos (`OrderService`).

**2. Refactorización Arquitectónica:** Para romper el ciclo de dependencias y aplicar DIP:

1. El `PdfGenerator` debe ser un componente utilitario puro. No debe conocer qué es un pedido ni llamar a ningún servicio de negocio; solo debe recibir una estructura de datos abstracta (`DocumentTemplate` o un mapa de datos a renderizar).
2. Se define una interfaz `IInvoiceService` en el paquete de pedidos o una abstracción compartida.
3. Las dependencias se inyectan a través de los constructores.

```
// 1. Modulo utilitario puro de PDF (Sin dependencias externas)
class PdfGenerator {
  renderDocument(title, metadata, content) {
    return `PDF [${title}]: ${JSON.stringify(content)}`;
  }
}

// 2. Modulo de Facturas (Depende unicamente del generador utilitario)
class InvoiceService {
  constructor(documentRenderer) {
    this.renderer = documentRenderer;
  }
  generateInvoice(invoiceDto) {
    return this.renderer.renderDocument("FACTURA", {}, invoiceDto);
  }
}

// 3. Modulo de Pedidos (Nivel de aplicacion - orquesta el flujo)
class OrderService {
  constructor(invoiceService) {
    this.invoiceService = invoiceService;
  }
  checkout(order) {
    const customer = this.lookupCustomer(order.id);
    const invoiceDto = {
      orderId: order.id,
      customer: customer,
```

```
      total: order.amount
    };
    return this.invoiceService.generateInvoice(invoiceDto);
  }
  lookupCustomer(orderId) {
    return "Cliente URJC";
  }
}
```

Listing 1.5: Código refactorizado limpio y sin ciclos

El nuevo grafo es estrictamente acíclico: OrderService ⟶ InvoiceService ⟶ PdfGenerator. Las dependencias fluyen en un único sentido sin retorno.

### ★ Ejercicio Resuelto: Evaluación Métrica de Cohesión y Acoplamiento

**Enunciado:**

Un equipo de desarrollo presenta dos alternativas para estructurar un módulo de comercio electrónico:

- **Diseño A:** Una única clase `ShopManager` que agrupa 25 métodos: cálculo de precios, conexión a base de datos Oracle, envío de emails mediante SMTP, parseo de ficheros XML de proveedores y renderizado de páginas HTML.
- **Diseño B:** Cuatro clases especializadas (`PricingEngine`, `OrderRepository`, `EmailNotificationService`, `CatalogImportService`) que se comunican mediante interfaces y DTOs bien definidos.

Compare cuantitativa y cualitativamente ambos diseños analizando acoplamiento, cohesión, coste de cambio y testeabilidad.

**Solución Razonada:**

| Dimensión de Análisis | Diseño A (God Class / Monolito) | Diseño B (Modular / Cohesivo) |
|---|---|---|
| **Nivel de Cohesión** | **Cohesión Coincidente/Lógica (Pésima).** Los métodos no comparten atributos; el índice LCOM es extremadamente elevado. | **Cohesión Funcional (Óptima).** Cada clase persigue un único objetivo atómico de negocio. |
| **Nivel de Acoplamiento** | **Alto Acoplamiento de Contenido y Común.** Cualquier consumidor de la tienda depende directa o indirectamente de la base de datos y de la librería SMTP. | **Bajo Acoplamiento de Datos.** Comunicación mediante contratos e interfaces desacopladas. |
| **Coste de Cambio (Mantenibilidad)** | **Altísimo.** Cambiar el driver de la base de datos o el proveedor de correo exige modificar y recompilar toda la clase `ShopManager`, arriesgando romper el cálculo de precios. | **Mínimo.** Los cambios en el sistema de correo quedan confinados en `EmailNotificationService` sin impacto colateral. |
| **Testeabilidad Unitario** | **Nula o muy deficiente.** Requiere configurar Oracle y un servidor de correo para probar un descuento del 10 %. | **Excelente.** Se prueban las reglas de precio en microsegundos con dobles de prueba (Mocks). |

## 1.6 Resumen Ejecutivo y Preguntas de Autoevaluación

**Resumen Sintético de Conceptos Clave**

- **Arquitectura Software:** Representa el diseño al nivel más alto de abstracción. Se formaliza mediante la tripleta de Perry & Wolf: **AS** = {Componentes, Forma, Lógica}.
- **Erosión vs. Mismatch:** El *Architectural Mismatch* surge por asunciones implícitas incompatibles al integrar componentes; la *Erosión Arquitectónica* es la degradación gradual de calidad por violar las directrices de diseño durante el mantenimiento.
- **Modelo vs. Arquitectura de Referencia:** El modelo es una división funcional abstracta (e.g., compilador de $N$ fases o modelo OSI); la arquitectura de referencia asocia dicho modelo a componentes software concretos y protocolos (e.g., AUTOSAR en la automoción).
- **Líneas de Producto (SPL):** Maximizan la reutilización compartiendo activos comunes (*commonalities*) y gestionando la variabilidad (*variability*) para derivar productos específicos.
- **Cohesión y Acoplamiento:** Regla de oro del diseño: **Alta Cohesión interna** (elementos de un módulo trabajan juntos hacia un fin único) y **Bajo Acoplamiento externo** (dependencias mínimas y abstractas entre módulos).
- **DIP vs. IoC vs. DI:**
  - **IoC:** Principio arquitectónico general (Principio de Hollywood: el framework toma el control del flujo).
  - **DIP:** Principio de diseño OO: módulos de alto nivel y bajo nivel dependen de abstracciones.
  - **DI:** Patrón y mecanismo técnico: un inyector suministra los servicios al cliente (preferentemente por constructor), facilitando tests unitarios independientes con mocks.

### 1.6.1 Batería de Preguntas de Autoevaluación (Estilo Examen URJC)

1. **¿Por qué la "lógica" (*rationale*) es un elemento indispensable en la definición de Perry y Wolf?**
   *Respuesta orientativa:* Porque documenta el porqué y las justificaciones técnicas de las decisiones de diseño tomadas frente a otras alternativas. Sin la lógica, los futuros ingenieros no pueden saber si una modificación en el sistema vulnerará los atributos de calidad originalmente requeridos.

2. **Explique la diferencia fundamental entre un Modelo de Referencia y una Arquitectura de Referencia, citando un ejemplo de cada uno.**
   *Respuesta orientativa:* El modelo de referencia define una partición funcional y flujos de datos abstractos sin ligarse a componentes software concretos (ejemplo: las 7 capas OSI o el esquema conceptual de un compilador). La arquitectura de referencia mapea ese modelo conceptual sobre componentes, conectores y estilos arquitectónicos reales (ejemplo: la arquitectura AUTOSAR para automoción).

3. **¿En qué consiste el fenómeno de la "erosión arquitectónica" y qué medidas preventivas puede implantar un equipo de ingeniería para evitarla?**
   *Respuesta orientativa:* Es la degradación paulatina de la calidad y de los límites del diseño software producida por cambios rápidos, parches y violaciones de encapsulamiento durante la fase de mantenimiento. Se previene automatizando pruebas unitarias de arquitectura en la integración continua con herramientas como ArchUnit o linters de dependencias modulares.

4. **¿Por qué el anti-patrón God Class conduce a un código difícilmente mantenible y cómo afecta a la cohesión?**
   *Respuesta orientativa:* Porque centraliza un número excesivo de responsabilidades no relacionadas, lo que desploma la cohesión funcional hacia niveles coincidentes o lógicos y dispara la complejidad ciclomática. Además, acopla fuertemente el sistema a dicha clase, impidiendo la realización de pruebas unitarias aisladas y provocando colisiones de código continuas entre desarrolladores.

5. **Demuestre cómo el Principio de Inversión de Dependencias (DIP) invierte el sentido de las flechas de dependencia del código fuente respecto al flujo de ejecución.**
   *Respuesta orientativa:* En tiempo de ejecución, el flujo de control se dirige desde el módulo de alto nivel

(cliente) hacia el módulo de bajo nivel (servicio). Sin embargo, al introducir una interfaz que pertenece conceptualmente al módulo de alto nivel, el código fuente del módulo de bajo nivel debe importar e implementar dicha interfaz. Por tanto, la flecha de dependencia estática del código fuente apunta hacia el módulo de alto nivel (hacia la abstracción), invirtiéndose respecto al flujo de control.

6. **Compare las tres modalidades de Inyección de Dependencias (Constructor, Setter, Interfaz) y justifique por qué la inyección por constructor es la más recomendada en entornos profesionales.**
   *Respuesta orientativa:* La inyección por constructor exige que todas las dependencias requeridas se proporcionen en el instante de la instanciación. Esto impide que existan objetos en estados intermedios inconsistentes, previene errores de puntero nulo (`NullPointerException`), favorece la inmutabilidad de los campos (declarables como `final` o `readonly`) y expone de forma explícita el contrato del componente. La inyección por setter solo es aconsejable para dependencias puramente opcionales.

# CAPÍTULO 2

# Estilos Arquitectónicos, Patrones de Diseño y Modelado con UML y C4

**Objetivos de Aprendizaje del Tema 2**

Al finalizar este capítulo, el estudiante e ingeniero de software será capaz de:

- Comprender y contrastar con rigor la **jerarquía de granularidad**: Estilos Arquitectónicos (macro-estructura) frente a Patrones de Diseño GoF (micro-estructura de clases) y su composición heterogénea en sistemas reales.
- Dominar el catálogo de los principales **Estilos Arquitectónicos**: Tuberías y Filtros (*Pipes & Filters*), Arquitectura por Capas (*Layers*), Cliente-Servidor, MVC, REST / RESTful (los 6 principios de Roy Fielding), Arquitecturas Dirigidas por Eventos (*EDA*), Pizarra (*Blackboard*) y Microservicios.
- Diseñar y resolver problemas de alta concurrencia y consistencia en sistemas distribuidos mediante patrones avanzados de microservicios: **SAGA** (coreografía vs. orquestación y transacciones compensatorias), **CQRS**, **Circuit Breaker**, **API Gateway** y **Database-per-Service**.
- Aplicar con solvencia el catálogo clásico de **Patrones de Diseño GoF**: Creacionales (*Singleton, Builder, Factory Method, Abstract Factory, Prototype*), Estructurales (*Adapter, Decorator, Facade, Proxy, Composite*) y de Comportamiento (*Strategy, State, Observer, Publish-Subscribe, Mediator, Chain of Responsibility, Visitor, Parallel Split*).
- Modelar arquitecturas de software complejas en **UML 2.5**: diagramas de paquetes con relaciones de dependencia e importación (`«import»`, `«uses»`), diagramas de clases, diagramas de componentes, diagramas de despliegue, máquinas de estados y secuencias.
- Utilizar los mecanismos formales de extensión de UML: estereotipos (`«stereotype»`), valores etiquetados, restricciones en **OCL** y perfiles especializados (**SysML** y **MARTE**).
- Dominar la **Notación C4** de Simon Brown: Contexto del Sistema, Contenedores, Componentes y Código, evaluando sus ventajas comunicativas y complementariedad frente a UML.

## 2.1 Estilos Arquitectónicos frente a Patrones de Diseño: Granularidad e Historia

### 2.1.1 Diferenciación Conceptual y Niveles de Granularidad

En el diseño de sistemas software existe una jerarquía fundamental de abstracción. Aunque tanto los estilos como los patrones son **soluciones probadas y reutilizables a problemas recurrentes en un contexto determinado**, difieren radicalmente en su **alcance, granularidad y momento de decisión**.

**▶ Concepto Clave: Estilo Arquitectónico (*Architectural Style*)**

Un **Estilo Arquitectónico** es un conjunto coordinado de decisiones de diseño que impone una **forma estructural y una organización de alto nivel** a un sistema software completo o a un subsistema principal. Define la familia de componentes permitidos, los tipos de conectores entre ellos, las restricciones topológicas de comunicación y los atributos de calidad fundamentales (rendimiento, escalabilidad, modificabilidad).

**▶ Concepto Clave: Patrón de Diseño Software (*Design Pattern*)**

Un **Patrón de Diseño** es una solución arquitectónica de **granularidad media-baja** orientada a resolver un problema específico de interacción, instanciación o composición dentro de un conjunto reducido de clases, interfaces y objetos colaboradores en el código fuente.

| Criterio | Estilo Arquitectónico | Patrón de Diseño (GoF / Fowler) |
|---|---|---|
| **Nivel de Abstracción** | Nivel Macro (Sistema global / Subsistemas). | Nivel Micro / Medio (Clases, objetos, módulos locales). |
| **Momento de Decisión** | Primeras fases de ingeniería (altísimo coste de cambio). | Durante la implementación y diseño detallado. |
| **Entidades involucradas** | Servidores, capas, tuberías, procesos, bases de datos. | Clases, interfaces, funciones, llamadas a método. |
| **Impacto Principal** | Atributos de calidad globales (escalabilidad, tolerancia a fallos). | Flexibilidad del código, mantenibilidad, testeabilidad local. |
| **Ejemplos Representativos** | Microservicios, Pipes & Filters, Event-Driven, REST. | Strategy, Factory Method, Observer, Adapter, Builder. |

Tabla 2.1: Comparativa rigurosa: Estilos Arquitectónicos frente a Patrones de Diseño.

**⇌ Compromiso Arquitectónico (*Trade-off*): Granularidad: Estilos vs. Patrones**

En el ciclo de diseño, **primero se selecciona el estilo (o estilos) arquitectónicos** para estructurar las áreas funcionales del sistema. Si no existiera un estilo definido, la solución degeneraría en una masa caótica de clases desordenadas. Una vez fijados los estilos macro, se seleccionan los patrones de diseño para estructurar los componentes internos.

**▲ ¡Atención / Ingeniería!: Heterogeneidad de las Arquitecturas del Mundo Real**

Una arquitectura software en producción **nunca es puramente monogenérica**: es una composición heterogénea donde suele predominar un estilo macro, dentro del cual subsistemas concretos adoptan otros estilos secundarios y cientos de patrones de diseño locales. Por ejemplo, una plataforma bancaria puede adoptar un estilo general de *Microservicios*, donde la comunicación externa sea *RESTful*, la comunicación asíncrona entre servicios sea *Event-Driven*, y el servicio de cálculo de riesgos emplee un estilo de *Pipes & Filters* codificado internamente con patrones *Strategy* y *Builder*.

### 2.1.2 Evolución Cronológica del Movimiento de Patrones

Para comprender la madurez de la disciplina es indispensable conocer sus hitos históricos:

1. **1977–1979 (Christopher Alexander):** El arquitecto civil Christopher Alexander publica *"A Pattern Language"* y *"The Timeless Way of Building"*, acuñando la célebre definición: *«Cada patrón describe un problema que ocurre una y otra vez en nuestro entorno, para describir después el núcleo de la solución a dicho problema, de tal forma que dicha solución pueda usarse millones de veces sin hacer nunca lo mismo dos veces»*.
2. **1987 (Ward Cunningham y Kent Beck):** Presentan en OOPSLA el primer lenguaje de patrones aplicado al diseño de interfaces de usuario en el lenguaje Smalltalk.
3. **1994–1995 (Gang of Four - GoF):** Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides publican el libro más influyente en la historia del software orientado a objetos: *"Design Patterns: Elements of Reusable Object-Oriented Software"*, catalogando 23 patrones clásicos clasificados en Creacionales, Estructurales y de Comportamiento.
4. **1996–2000 (Frank Buschmann et al. - POSA):** La serie *"Pattern-Oriented Software Architecture"* introduce los patrones arquitectónicos de alto nivel (como Broker, Pipes and Filters, Blackboard).
5. **2002 (Martin Fowler):** Publica *"Patterns of Enterprise Application Architecture"*, estableciendo las bases para capas de dominio, persistencia (Active Record, Data Mapper) y gestión de transacciones.
6. **2014–Presente:** Patrones distribuidos, Cloud-Native, microservicios (Chris Richardson) y los emergentes patrones de IA generativa (RAG, Agentes autónomos, Function Calling).

## 2.2 Catálogo Exhaustivo de Estilos Arquitectónicos

### 2.2.1 Tuberías y Filtros (*Pipes and Filters*)

El estilo de Tuberías y Filtros descompone el procesamiento de un flujo continuo de datos en una secuencia de etapas de transformación independientes.

**◊ Patrón / Estilo Arquitectónico: Tuberías y Filtros (*Pipes & Filters*)**

**Problema:** Transformar incrementalmente flujos continuos de datos mediante etapas secuenciales independientes sin acoplar los pasos de computación entre sí.
**Solución:**

- **Filtro (*Filter*):** Componente de caja negra que lee flujos de datos de entrada, transforma la información de manera incremental y emite flujos de salida. Comienza a producir datos antes de que termine de leer toda la entrada.
- **Tubería (*Pipe*):** Conector unidireccional que transporta el flujo de datos entre la salida de un filtro y la entrada del siguiente, gestionando la sincronización y la memoria intermedia.




Figura 2.1: Flujo secuencial del estilo Pipes and Filters.

**✓ Ejemplo Práctico: Ejemplos en Sistemas Reales**

- **Tuberías en sistemas Unix/Linux:** El clásico encadenamiento de comandos de consola:

```
ls -al | grep "fichero" > salida.txt
cat access.log | grep "500" | awk '{print $1}' | sort | uniq -c | sort -nr
```

```
3
```

- **Pipelines de Procesamiento Multimedia:** Compresión de vídeo en FFmpeg o procesamiento de audio en tiempo real.
- **ETL (Extract-Transform-Load) en Big Data:** Apache Spark Streaming o Apache Kafka Streams.

**⇌ Compromiso Arquitectónico (*Trade-off*): Pipes and Filters**

- **Ventajas:** Reusabilidad absoluta de filtros independientes, paralelismo natural (ejecución concurrente en múltiples núcleos), sencillez de pruebas unitarias.
- **Desventajas:** Sobrecarga computacional por parseo, formateo y serialización continua de datos entre tuberías; pésimo para aplicaciones interactivas GUI.

### 2.2.2 Estilo por Capas (*Layered Architecture*)

Organiza el sistema en una jerarquía de niveles horizontales ordenados, donde cada capa soporta una API de servicios para que sea utilizada exclusivamente por la capa superior.

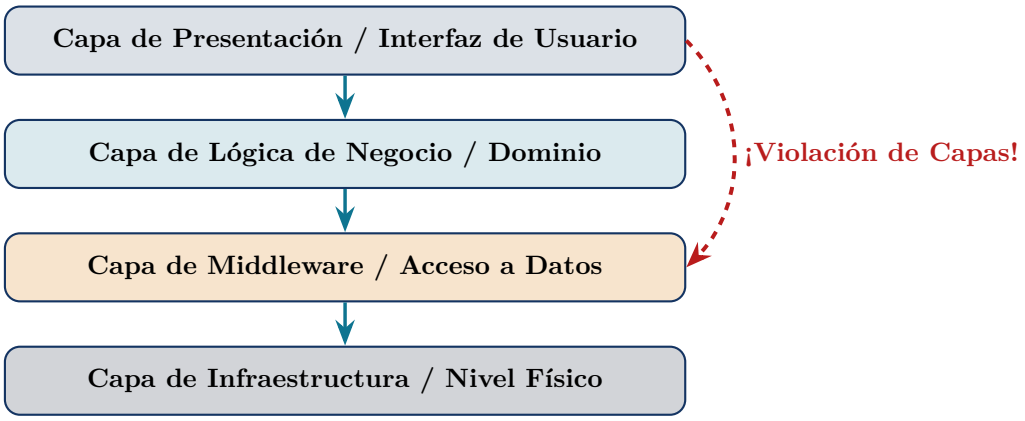



Figura 2.2: Arquitectura en capas estricta frente a atajos ilegítimos que violan el diseño.

**▲ ¡Atención / Ingeniería!: Reglas de Diseño en Capas**

- **Capas Estrictas (*Strict Layers*):** La capa $N$ sólo invoca a la capa inmediatamente inferior $N-1$.
- **Capas Horizontales y Verticales:** Además de capas horizontales apiladas, existen subsistemas transversales que proporcionan soporte a todas las capas (como la **Capa de Seguridad**, **Logging** o **Librerías compartidas**).
- **Violación del Diseño entre Capas:** Ocurre cuando un elemento de la capa de interfaz llama directamente al middleware o a la base de datos sin pasar por la lógica de negocio, violando el encapsulamiento y eludiendo las reglas de validación y auditoría.

### 2.2.3 Estilo Cliente-Servidor (*Client-Server*)

Modelo fundamental de Internet y redes distribuidas:

- Los clientes acceden a datos y aplicaciones centralizadas en los servidores.
- Arquitectura basada en peticiones y respuestas síncronas o asíncronas con eventos.

- Evolución hacia arquitecturas cliente-servidor distribuidas (redes Peer-to-Peer).

### 2.2.4 Modelo-Vista-Controlador (*MVC*)

Surgido en los años 80 para sistemas interactivos con interfaces gráficas, permite disponer de **diferentes vistas de los mismos datos** y habilita la interacción multimodal:

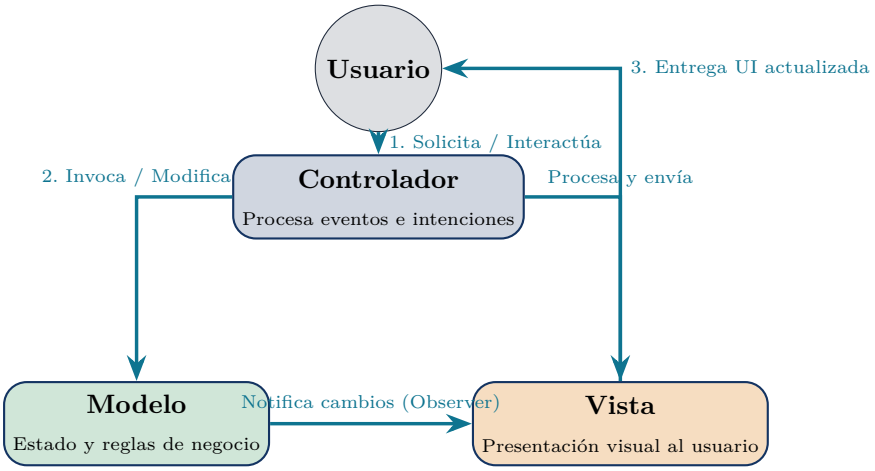



Figura 2.3: Ciclo de colaboración en el estilo Modelo-Vista-Controlador (MVC).

▶ **Concepto Clave: Componentes del MVC**

- **Modelo (*Model*):** Representa los datos del dominio, el estado y las reglas de negocio. Notifica a las vistas cuando su estado muta (habitualmente mediante el patrón *Observer*).
- **Vista (*View*):** Renderiza los datos del modelo en un formato interactivo para el usuario (layout y lógica de presentación).
- **Controlador (*Controller*):** Intercepta las acciones del usuario y las traduce en comandos sobre el modelo.

### 2.2.5 El Estilo REST y Arquitecturas RESTful

En el año 2000, Roy Thomas Fielding presentó en su tesis doctoral la definición formal de **REST** (*Representational State Transfer*) para formalizar el diseño arquitectónico de servicios web e Internet.

★ **Principio Fundamental de Diseño: Los 6 Principios Arquitectónicos de Roy Fielding para REST**

Para que un sistema sea estrictamente **RESTful**, debe satisfacer seis restricciones:

1. **Cliente-Servidor (C/S):** Separación tajante entre interfaz de usuario y almacenamiento de datos.
2. **Sin Estado (*Statelessness*):** Cada petición contiene todo el contexto necesario. El cliente almacena el estado de sesión; el servidor nunca retiene estado de cliente.
3. **Capacidad de Almacenamiento en Caché (*Cacheability*):** Respuestas etiquetadas explícitamente para reutilización local o intermedia.
4. **Sistema en Capas (*Layered System*):** Intermediarios transparentes (proxies, balanceadores, firewalls).
5. **Interfaz Uniforme (*Uniform Interface*):** URIs estándar para recursos, manipulación vía representaciones (JSON/XML) y mensajes autodescriptivos.
6. **Código bajo demanda (*Code on Demand* - Opcional):** Envío de scripts ejecutables (e.g.,

JavaScript) al cliente.

**Atributos de Calidad que optimiza REST:**

- **Rendimiento (*Performance*):** Minimizando latencias gracias al almacenamiento masivo en caché distribuida.
- **Escalabilidad (*Scalability*):** Dado que el servidor es *stateless*, cualquier réplica puede atender cualquier petición.
- **Simplicidad (*Simplicity*):** Interfaz uniforme basada en URIs universales y verbos HTTP estándar.
- **Modificabilidad (*Modifiability*):** Evolución independiente de clientes y servidores.
- **Fiabilidad (*Reliability*):** Alta resistencia a caídas de nodos individuales.

### 2.2.6 Arquitecturas Dirigidas por Eventos (*Event-Driven Architecture - EDA*)

En una arquitectura EDA, los componentes se comunican mediante la **detección, consumo y reacción a eventos**:

**► Concepto Clave: Evento de Software**

Un **evento** es un registro inmutable que atestigua un cambio significativo en el estado de un sistema ocurrido en el pasado. Los emisores de eventos **no conocen a los consumidores**, asegurando un desacoplamiento temporal y espacial radical (*loose coupling*).

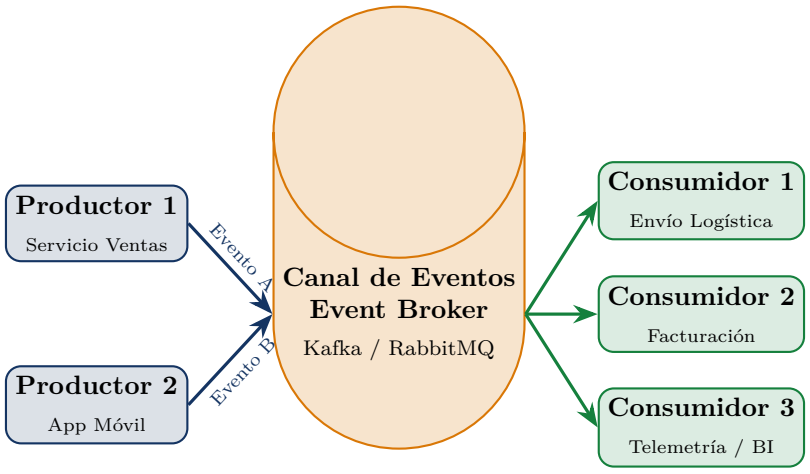



Figura 2.4: Topología genérica de una Arquitectura Dirigida por Eventos (EDA) con un Event Broker central.

**Formas de Procesar Eventos:**

1. **Eventos Simples:** Cambios medibles discretos en una condición (e.g., un sensor que detecta exceso de temperatura).
2. **Eventos en "Stream":** Cadenas continuas de eventos enviadas en tiempo real a suscriptores.
3. **Eventos Complejos (CEP):** Confluencia y correlación de patrones entre múltiples eventos temporales (e.g., detección de fraude bancario multidivisa).

### 2.2.7 Otros Estilos Notables: Pizarra (*Blackboard*) y Peer-to-Peer

- **Estilo Pizarra (*Blackboard*):** Especialmente útil en problemas de inteligencia artificial y reconocimiento de patrones donde no existe un algoritmo determinista para hallar la solución directa. Se compone de:
  - *La Pizarra (*Blackboard*):* Memoria de trabajo compartida que alberga el estado actual del problema y las soluciones parciales.
  - *Fuentes de Conocimiento (*Knowledge Sources*):* Agentes especialistas autónomos que vigilan la pizarra y aportan soluciones parciales cuando identifican información relevante.
  - *Mecanismo de Control:* Decide qué fuente de conocimiento tiene el turno para actuar.
- **Estilo Peer-to-Peer (P2P):** Descentralización absoluta. Cada nodo de la red actúa simultáneamente como cliente y servidor (*servent*), compartiendo ancho de banda, capacidad de cómputo y almacenamiento (e.g., BitTorrent, redes blockchain de Bitcoin y Ethereum).

## 2.3 Patrones Arquitectónicos para Sistemas Distribuidos y Microservicios

La transición desde el monolito hacia la **Arquitectura de Microservicios (MSA)** exige un conjunto avanzado de patrones para garantizar la consistencia, disponibilidad y resiliencia del sistema distribuido.

**⇌ Compromiso Arquitectónico (*Trade-off*): Microservicios vs. Monolito Modular**

- **Microservicios:** Despliegues independientes, escalabilidad granular, heterogeneidad tecnológica y alta autonomía de equipos. A cambio, introducen complejidad operativa (DevOps masivo, observabilidad distribuida, latencia de red y riesgo de fallo parcial).
- **Monolito Modular:** Máximo rendimiento en llamadas en memoria, sencillez de transacciones ACID y facilidad de pruebas iniciales. A cambio, límites de escalabilidad organizativa en equipos de cientos de ingenieros.

**[X] Anti-Patrón de Diseño: Monolito Distribuido (*Distributed Monolith*)**

Ocurre cuando un equipo divide su sistema en decenas de microservicios pero mantiene acoplamiento síncrono HTTP estricto entre ellos o una base de datos única compartida. Se obtienen todas las desventajas de los microservicios (latencia de red, fallos en cascada, pesadilla de despliegue) sin ninguna de sus ventajas (despliegues independientes, autonomía).

### 2.3.1 El Patrón SAGA: Gestión de Transacciones Distribuidas

En una arquitectura con base de datos por servicio (*Database-per-Service*), no es posible utilizar transacciones ACID tradicionales de base de datos relacional (como el protocolo pesado de bloqueo en dos fases *Two-Phase Commit - 2PC*) debido a su bajísimo rendimiento y riesgo de bloqueo global.

**◊ Patrón / Estilo Arquitectónico: SAGA (Transacciones Distribuidas Compensatorias)**

**Problema:** ¿Cómo mantener la consistencia de datos entre múltiples microservicios con bases de datos independientes sin usar bloqueos distribuidos?
**Solución:** Implementar cada transacción de negocio como una secuencia de transacciones locales coordinadas. Cada transacción local actualiza la base de datos y emite un mensaje/evento. Si un paso intermedio falla, la SAGA ejecuta una serie de **transacciones compensatorias** que deshacen los cambios en orden inverso.




Figura 2.5: Flujo de ejecución de una SAGA con transacciones compensatorias ante fallo.

**▲ ¡Atención / Ingeniería!: Modelos de Coordinación de SAGAs**

Existen dos formas de coordinar una SAGA:

1. **Coreografía (*Choreography*):** Descentralizada. Cada servicio escucha eventos del bus y decide por sí mismo cuándo ejecutar su transacción local y qué evento emitir. No hay un punto único de control. Es óptima para flujos simples (2 a 4 pasos), pero difícil de depurar cuando el número de participantes crece.
2. **Orquestación (*Orchestration*):** Centralizada. Un componente dedicado (**Saga Orchestrator**) actúa como director de orquesta enviando comandos directos a los microservicios participantes y procesando sus respuestas. Centraliza la máquina de estados de la transacción, simplificando la observabilidad y el control.

### 2.3.2 CQRS (*Command Query Responsibility Segregation*)

En muchos sistemas de alta demanda, los modelos conceptuales requeridos para **escribir datos** son completamente dispares a los requeridos para **leer datos**.

**◊ Patrón / Estilo Arquitectónico: CQRS (Command Query Responsibility Segregation)**

**Problema:** ¿Cómo implementar una consulta compleja que requiere agregar datos dispersos en múltiples microservicios sin sobrecargar los servicios ni violar el encapsulamiento de datos?
**Solución:** Diseñar una vista de base de datos réplica o almacén de lectura optimizado exclusivamente para soportar las consultas. La aplicación mantiene sincronizada esta réplica mediante suscripción a eventos de dominio emitidos por cada servicio propietario de los datos.

Plataformas industriales como **Eventuate** (*eventuate.io*) proporcionan infraestructura completa para la gestión de datos distribuida con soporte nativo para eventos y CQRS.

### 2.3.3 Circuit Breaker (Disyuntor)

**◊ Patrón / Estilo Arquitectónico: Circuit Breaker (Tolerancia a Fallos y Resiliencia)**

**Problema:** Las peticiones a servicios remotos pueden fallar o demorarse indefinidamente por saturación de red, agotando los hilos de los servicios llamantes y provocando una caída catastrófica en cascada.
**Solución:** Envolver las llamadas remotas en un disyuntor que monitoriza la tasa de fallos:

- **Cerrado (*Closed*):** Operación normal. Si la tasa de fallos supera el umbral límite, cambia a **Abierto**.
- **Abierto (*Open*):** Falla inmediatamente en local (*Fail Fast*) sin invocar al servicio remoto, retornando un fallback.

- **Semi-Abierto (*Half-Open*):** Tras un tiempo de espera, permite pasar algunas peticiones de prueba para verificar si el servicio remoto se ha recuperado.

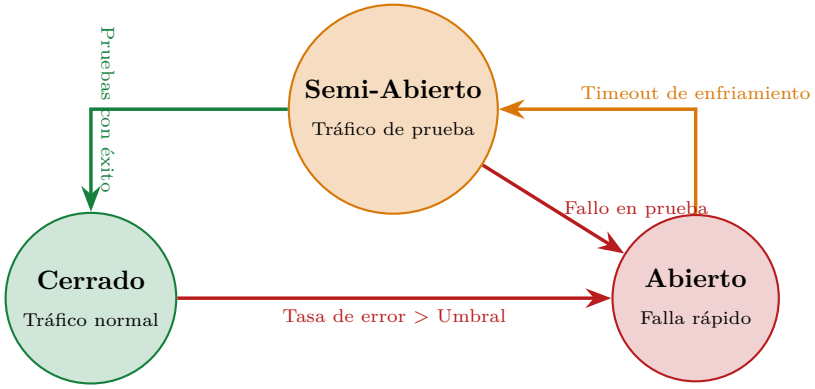



Figura 2.6: Máquina de estados del patrón Circuit Breaker.

## 2.4 Patrones de Diseño Clásicos (GoF)

Los 23 patrones clásicos del libro de Gamma et al. (Gang of Four) se estructuran en tres grandes categorías según su propósito fundamental.

### 2.4.1 Patrones Creacionales

Abstraen el proceso de instanciación de objetos, desvinculando al sistema de cómo se crean, componen y representan sus elementos.

1. **Singleton:**
   **Propósito:** Garantiza que una clase posea una **única instancia** en todo el ciclo de vida de la aplicación y proporciona un punto de acceso global a dicha instancia.

```
public class DatabaseConnectionPool {
    private static volatile DatabaseConnectionPool instance;

    private DatabaseConnectionPool() {
        // Constructor privado para impedir 'new' desde el exterior
    }

    public static DatabaseConnectionPool getInstance() {
        if (instance == null) {
            synchronized (DatabaseConnectionPool.class) {
                if (instance == null) {
                    instance = new DatabaseConnectionPool();
                }
            }
        }
        return instance;
    }
}

```

Listing 2.1: Implementación Thread-Safe del patrón Singleton con Double-Checked Locking

2. **Builder:**
   **Propósito:** Abstrae el proceso de creación de un objeto complejo, centralizando dicho proceso en un único punto y permitiendo crear diferentes representaciones con el mismo proceso de construcción.

3. **Factory Method:**
   **Propósito:** Centraliza en una clase constructora la creación de objetos de un subtipo determinado, ocultando la diversidad de casos particulares y permitiendo a las subclases modificar el tipo de objeto a crear.
4. **Abstract Factory:**
   **Propósito:** Encapsula un grupo de fábricas individuales bajo un tema común y utiliza interfaces genéricos para crear **familias de objetos relacionados** sin especificar sus clases concretas.
5. **Prototype:**
   **Propósito:** Crea nuevos objetos duplicándolos o clonándolos a partir de una instancia prototípica existente.

### 2.4.2 Patrones Estructurales

Se centran en cómo se componen y ensamblan las clases y los objetos para formar estructuras más complejas y flexibles.

1. **Adapter (Wrapper):**
   **Propósito:** Transforma una interfaz en otra. Adapta una interfaz para que pueda ser utilizada por una clase que de otro modo no podría utilizarla (e.g., wrappers para sistemas bancarios antiguos en COBOL dotándolos de un front-end Web moderno).
2. **Facade (Fachada):**
   **Propósito:** Provee una interfaz unificada y simplificada a una librería, un framework o un conjunto complejo de clases de un subsistema.
3. **Proxy:**
   **Propósito:** Proporciona un intermediario de un objeto para controlar su uso (habitual en servidores web para hacer caché de páginas o conexiones remotas).
4. **Decorator:**
   **Propósito:** Añade funcionalidad a una clase de forma dinámica en tiempo de ejecución.

### 2.4.3 Patrones de Comportamiento

Abordan la asignación de responsabilidades, los algoritmos y la comunicación dinámica entre objetos.

1. **Observer (Publicador-Suscriptor Local):**
   **Propósito:** Define una dependencia de uno-a-muchos entre objetos de forma que, cuando el sujeto cambia su estado, todos sus observadores son notificados y actualizados automáticamente.
2. **Publish-Subscribe:**
   **Propósito:** Variante desacoplada a través de red donde los remitentes envían mensajes a canales sin conocer la identidad ni el número de suscriptores (MOM - Middleware Orientado a Mensajes).
3. **Mediator:**
   **Propósito:** Encapsula la comunicación entre objetos a través de un objeto mediador central, reduciendo el acoplamiento y dependencias directas entre componentes.
4. **Strategy:**
   **Propósito:** Permite disponer de varios métodos para resolver un problema y elegir cuál utilizar en tiempo de ejecución.
5. **State:**
   **Propósito:** Permite que un objeto modifique su comportamiento cada vez que cambia su estado interno.
6. **Chain of Responsibility:**
   **Propósito:** Permite establecer una línea o cadena que deben recorrer los mensajes para que los objetos gestionen la petición secuencialmente.

7. **Visitor:**
   **Propósito:** Define nuevas operaciones sobre una jerarquía de clases sin modificar las clases originales sobre las que opera.

### 2.4.4 Otros Patrones: Parallel Split (Flujos de Trabajo / Workflows)

**♢ Patrón / Estilo Arquitectónico: Parallel Split**

**Propósito:** Es un punto del flujo de control donde un hilo de ejecución individual se bifurca en múltiples hilos que pueden ejecutarse concurrentemente en paralelo, permitiendo la ejecución simultánea de varias actividades. Es un patrón básico de control de flujo en orquestaciones web (BPEL, reservas de vuelos u hoteles donde se consultan múltiples aerolíneas a la vez).

## 2.5 Vistas Arquitectónicas y Modelado con UML 2.5

El estándar **UML 2.5** (Unified Modeling Language) de la OMG proporciona la notación gráfica formal para documentar las decisiones arquitectónicas en las distintas vistas del sistema.

### 2.5.1 El Modelo de Vistas 4+1 de Philippe Kruchten

**▶ Concepto Clave: Modelo 4+1 (Kruchten, 1995)**

Propone organizar la documentación arquitectónica en cinco vistas complementarias:

1. **Vista Lógica (Design View):** Captura las entidades lógicas (clases, paquetes, interfaces) y cómo se interconectan. Satisface los requisitos funcionales.
2. **Vista de Procesos (Process View):** Modela aspectos dinámicos de concurrencia, sincronización, hilos y rendimiento.
3. **Vista de Implementación / Desarrollo:** Modela los componentes de software físicos (artefactos, librerías, dependencias).
4. **Vista de Despliegue (Deployment View):** Modela la asignación de entidades lógicas y componentes sobre la infraestructura física de hardware y red.
5. **Vista de Casos de Uso (+1):** Escenarios que validan e integran las cuatro vistas anteriores.

### 2.5.2 Diagramas de Paquetes: Modelado de Subsistemas y Estilos

Los paquetes describen subsistemas. **No debe haber paquetes vacíos** y deben llevar un estereotipo (`«stereotype»`) que indique su misión:

### 2.5.3 Taxonomía de Relaciones en Diagramas de Clases UML

### 2.5.4 Mecanismos de Extensión de UML: Estereotipos, Valores Etiquetados y OCL

1. **Estereotipos (`«nombre»`):** Extienden la semántica de un elemento para representar una variante con distinta intención (e.g., `«exception»`, `«entity»`, `«boundary»`).
2. **Valores Etiquetados (*Tagged Values*):** Extienden las propiedades añadiendo pares clave-valor entre llaves: `{versión=3.2, autor=ÜRJC"}`.
3. **Restricciones y OCL (*Object Constraint Language*):** Notación formal para especificar pre-condiciones, post-condiciones e invariantes del sistema (e.g., `{edad > 18}`).

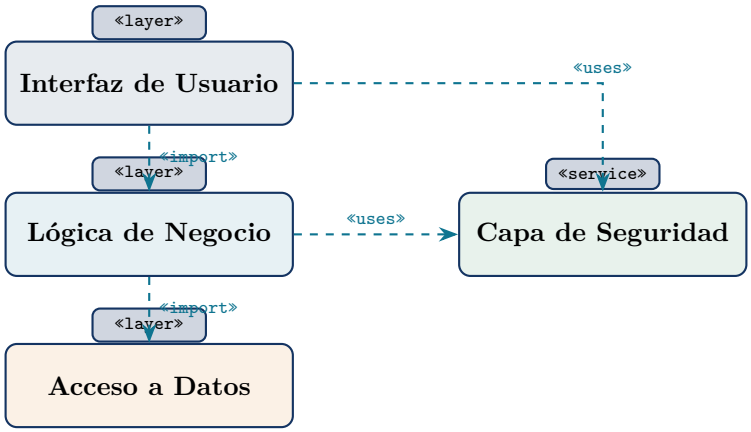



Figura 2.7: Modelado UML del estilo por capas mediante diagramas de paquetes.

| Relación UML | Simbología Visual | Significado Semántico y Ciclo de Vida |
|---|---|---|
| **Asociación** | Línea sólida continua (con flecha opcional) | Conexión semántica estructural entre clases. Indica envío de mensajes o conocimiento mutuo. |
| **Agregación Débil** | Línea con **rombo blanco / vacío** en el contenedor | Relación "todo-parte" no destructiva. La parte puede existir independientemente del contenedor (ej., Universidad y Estudiante). |
| **Composición Fuerte** | Línea con **rombo negro / relleno** en el contenedor | Relación de pertenencia física y ciclo de vida compartido. Si se destruye el contenedor, se destruyen obligatoriamente las partes (ej., Factura y LíneasDeFactura). |
| **Generalización** | Línea con **flecha triangular hueca** hacia la superclase | Herencia estricta en orientación a objetos. Relación taxonómica "es un tipo de". |
| **Realización** | Línea **discontinua con flecha triangular hueca** | Implementación de un contrato o interfaz abstracta por parte de una clase concreta. |
| **Dependencia** | Línea **discontinua con flecha simple** | Un cambio en el elemento proveedor puede forzar cambios en el cliente. Uso temporal como parámetro de método (`«uses»`). |

Tabla 2.2: Taxonomía formal de relaciones en diagramas de clases UML.

▲ **¡Atención / Ingeniería!: Perfiles Especializados: SysML y MARTE**

- **SysML (*Systems Modeling Language*):** Extensión de la paleta tradicional de símbolos UML para modelar ingeniería de sistemas (hardware, energía, requisitos).
- **MARTE (*Modeling and Analysis of Real-Time and Embedded Systems*):** Extensión oficial para sistemas empotrados y de tiempo real con soporte explícito para deadlines, prioridades y análisis de planificabilidad temporal.

## 2.6 Notación Arquitectónica Moderna: El Modelo C4

Creado por el arquitecto Simon Brown, el **Modelo C4** (*Context, Containers, Components, Code*) fue concebido para ayudar a los desarrolladores a comunicar la arquitectura y documentar el diseño del software reflejando fielmente cómo piensan los arquitectos.

▶ **Concepto Clave: Abstracciones del Modelo C4**

1. **System Context (Contexto):** Punto de partida. No muestra detalles internos; se centra en los usuarios (*stakeholders*) y los sistemas externos con los que interactúa el software.
2. **Container (Contenedores):** Muestra los contenedores principales del sistema (aplicaciones web SPA, apps móviles, microservicios, bases de datos). Incluye decisiones tecnológicas clave.
3. **Component (Componentes):** Amplía el detalle de un contenedor individual, mostrando sus componentes lógicos internos (controladores, servicios, fachadas).
4. **Code (Código):** Detalles de implementación a nivel de clases e interfaces (habitualmente diagramas de clases UML).

| Nivel C4 | Audiencia Objetivo | ¿Qué muestra? |
|---|---|---|
| **1. Contexto** | Negocio, directores, nuevos desarrolladores | El sistema completo como una caja negra en el centro, rodeado por los usuarios humanos y los sistemas externos con los que se comunica. |
| **2. Contenedores** | Desarrolladores, DevOps, arquitectos | La arquitectura de ejecución: aplicaciones web (SPA), backend API, base de datos Postgres, broker Kafka, sistema de archivos. |
| **3. Componentes** | Desarrolladores que implementan código | La descomposición interna de un contenedor concreto: módulos lógicos, interfaces internas y patrones. |
| **4. Código** | Ingenieros de implementación | Opcional. Representación exacta de clases y métodos (habitualmente diagramas de clases generados automáticamente). |

Tabla 2.3: Resumen de los cuatro niveles de abstracción en C4.

## 2.7 Ejercicios Resueltos y Casos Prácticos de Examen

★ **Ejercicio Resuelto: Diseño de una Transacción Distribuida con Patrón SAGA**

**Enunciado:**
Una plataforma de venta de entradas para conciertos en línea migró a microservicios: `OrderService`, `PaymentService` y `TicketInventoryService`. Se requiere procesar una compra garantizando que, si el cobro bancario falla tras haberse reservado las butacas, el sistema no quede en un estado inconsistente.

**Tarea del Ingeniero:**

1. Justifique por qué el protocolo Two-Phase Commit (2PC) no es adecuado para este escenario.
2. Diseñe una SAGA Orquestada detallando las transacciones locales y las transacciones compensatorias.

**Solución Razonada:**
**1. Inadecuación de Two-Phase Commit (2PC):** El protocolo 2PC es síncrono y bloqueante: durante la fase de votación y preparación, las filas de base de datos en todos los microservicios participantes quedan bloqueadas exclusivamente. En un evento de venta masiva de entradas con miles de usuarios concurrentes, 2PC colapsaría el rendimiento, reduciría la disponibilidad drásticamente y aumentaría el riesgo de caídas en cascada. Por ello, se aplica el principio de consistencia eventual mediante el patrón SAGA.
**2. Diseño de la SAGA Orquestada:**

- **Orquestador:** `CreateOrderSagaOrchestrator`.
- **Paso 1 ($T_1$):** `TicketInventoryService.bloquearButacas(eventoId, butacas)` → Éxito (las butacas pasan a estado `RESERVADA_TEMPORAL`).
- **Paso 2 ($T_2$):** `PaymentService.cobrarTarjeta(tarjetaId, importe)` → **¡Fallo: Tarjeta caducada o fondos insuficientes!**
- **Compensación ($C_1$):** El orquestador detecta el fallo en $T_2$ y envía el comando de compensación: `TicketInventoryService.liberarButacas(eventoId, butacas)`. Las butacas vuelven a estar disponibles inmediatamente para otros usuarios.
- **Cierre:** El orquestador marca el pedido en `OrderService` con estado `CANCELADO_POR_ERROR_PAGO`.

★ **Ejercicio Resuelto: Selección Justificada de Estilos Arquitectónicos**

**Enunciado:**
Una multinacional de logística requiere diseñar un sistema para rastrear en tiempo real 500.000 camiones de reparto en Europa. Cada camión envía su posición GPS y lecturas de telemetría del motor cada 5 segundos. La aplicación debe:

- Detectar anomalías térmicas en la cadena de frío en menos de 2 segundos.
- Generar informes de consumo mensual para gerencia.

Seleccione y razone qué estilo o combinación heterogénea de estilos arquitectónicos debe adoptar la solución.

**Solución Razonada:** Se requiere una combinación heterogénea de estilos:

1. **Estilo Dirigido por Eventos (EDA) con Event Streaming (Kafka/Pulsar):** Para la ingesta masiva de telemetría en tiempo real desde los 500.000 camiones (100.000 eventos/segundo) de forma desacoplada y tolerante a fallos.
2. **Procesamiento de Eventos Complejos (CEP) / Pipes & Filters:** Un motor de streaming continuo (Apache Flink) procesa el flujo en ventana deslizante para detectar la anomalía térmica en menos de 2 segundos y disparar alertas inmediatas.
3. **CQRS:** Se segrega el modelo de escritura (ingesta masiva de streaming en base de datos Time-Series) del modelo de consultas para los informes de gerencia (Data Warehouse analítico con consultas OLAP).

## 2.8 Resumen Ejecutivo y Preguntas de Autoevaluación

**Resumen de Conceptos Clave del Tema 2**

- **Estilos vs. Patrones:** Los estilos definen la estructura global y los grandes subsistemas; los patrones operan a escala micro/media sobre clases y objetos.
- **Principios RESTful:** Cliente-Servidor, Sin Estado (*Stateless*), Cacheable, Sistema en Capas, Interfaz Uniforme y Código bajo demanda opcional.
- **Microservicios Avanzados:** Se rigen por patrones como SAGA (transacciones compensatorias locales para consistencia eventual), CQRS (segregación de lectura y escritura) y Circuit Breaker (resiliencia y contención de fallos).
- **Modelado UML 2.5:** Los paquetes definen los estilos de la arquitectura; las relaciones entre paquetes deben usar `«import»` sin atravesar niveles de forma caótica.
- **Modelo C4:** Alternativa moderna y complementaria a UML basada en 4 niveles de zoom: Contexto, Contenedores, Componentes y Código.

### 2.8.1 Batería de Preguntas de Autoevaluación

1. **¿Por qué una arquitectura software no puede calificarse únicamente bajo un único estilo?**
   *Respuesta orientativa:* Porque los sistemas reales son heterogéneos: un estilo macro (como Microservicios) suele alojar subsistemas que implementan internamente otros estilos (como Pipes & Filters o EDA) para satisfacer atributos de calidad diversos.

2. **Explique los tres estados del patrón Circuit Breaker y cómo previenen las caídas en cascada.**
   *Respuesta orientativa:* Cerrado (tráfico normal), Abierto (falla rápido en local sin sobrecargar el servicio caído) y Semi-Abierto (prueba peticiones exploratorias tras un tiempo de enfriamiento). Impide que los hilos de los servicios llamantes se bloqueen esperando timeouts acumulados.

3. **¿Cuál es la diferencia semántica entre agregación y composición en un diagrama de clases UML?**
   *Respuesta orientativa:* La agregación (rombo blanco) es una relación débil donde las partes pueden subsistir sin el contenedor. La composición (rombo negro) es fuerte y define coincidencia de ciclo de vida: si se destruye el contenedor, las partes se destruyen obligatoriamente con él.

4. **¿Qué representan los cuatro niveles de la Notación C4?**
   *Respuesta orientativa:* 1. Contexto (visión general con personas y sistemas externos); 2. Contenedores (aplicaciones ejecutables y almacenes de datos); 3. Componentes (módulos internos de un contenedor); 4. Código (diagramas de clases de implementación).

# CAPÍTULO 3

# Decisiones de Diseño y Gestión del Conocimiento Arquitectónico

**Objetivos de Aprendizaje del Tema 3**

Al finalizar este capítulo, el estudiante e ingeniero de software será capaz de:

- Comprender el cambio de paradigma formulado por Jan Bosch (2004): la **Arquitectura Software como un conjunto de decisiones de diseño** (*Architectural Design Decisions - ADD*) más que como una mera composición de componentes y conectores.
- Identificar y mitigar el fenómeno de la **evaporación del conocimiento tácito** (*knowledge vaporization*) en las organizaciones de ingeniería.
- Aplicar el estándar internacional **ISO/IEC/IEEE 42010:2022**, reconociendo a las decisiones de diseño y a su justificación (*rationale*) como elementos de primera clase.
- Analizar e implementar la **Vista de la Decisión** (*Decision View*) como una dimensión arquitectónica transversal a los requisitos y a las vistas del modelo 4+1 de Kruchten.
- Dominar los procesos del ciclo de vida de la **Gestión del Conocimiento Arquitectónico** (*Architecture Knowledge Management - AKM*): Creación, Compartición, Evaluación y Aprendizaje.
- Establecer redes formales de **trazabilidad externa** (*forward* y *backward traceability*) entre requisitos, decisiones y elementos de diseño, así como de **trazabilidad interna** entre decisiones mediante ontologías.
- Conocer la evolución histórica de las tres generaciones de herramientas de AKM (desde *PAKME*, *Archium* y *ADDSS* hasta *SAW*, *ADvISE* y *ADMentor*).
- Redactar registros de decisión arquitectónica profesionales mediante el formato estándar **MADR** (*Markdown Architectural Decision Records*), evitando errores habituales de superficialidad y descontextualización.
- Aplicar el marco de pensamiento crítico y reflexivo **Mind 1 / Mind 2** para desafiar y validar decisiones arquitectónicas antes de consolidarlas en código.

## 3.1 Fundamentos del Conocimiento Arquitectónico y Decisiones de Diseño (ADD)

### 3.1.1 El Cambio de Paradigma: Jan Bosch (2004)

Tradicionalmente, la arquitectura software se describía casi en exclusiva mediante diagramas de cajas y flechas que representaban componentes computacionales y conectores de comunicación. Sin embargo, en 2004, el investigador Jan Bosch formuló una crítica fundamental que transformó la disciplina:

> ★ **Principio Fundamental de Diseño: La Arquitectura Software como Conjunto de Decisiones (Jan Bosch, 2004)**
>
> *«Una Arquitectura Software debe ser considerada primariamente como el resultado acumulado de un conjunto de Decisiones de Diseño Arquitectónico (ADD – Architectural Design Decisions), más que como un mero conjunto de componentes y conectores».*

Cuando únicamente se conservan los diagramas estructurales y el código resultante, se produce la denominada **evaporación del conocimiento arquitectónico** (*Architectural Knowledge Vaporization*):

- Se desconoce el **porqué** (*rationale*) de la estructura adoptada.
- Se olvidan las **alternativas descartadas** y los motivos específicos por los que fueron rechazadas.
- Se ignora qué requisitos no funcionales críticos justificaron la complejidad introducida.

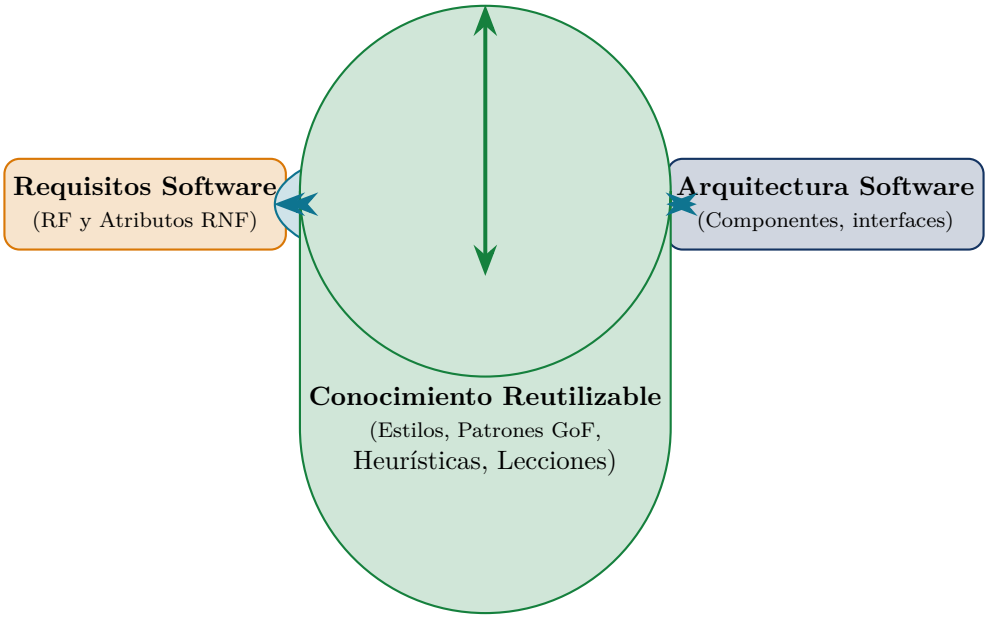



Figura 3.1: Las Decisiones de Diseño como puente integrador entre los requisitos y la arquitectura.

### 3.1.2 Conocimiento Tácito frente a Conocimiento Explícito

En la mayoría de las empresas de desarrollo, la mayor parte del conocimiento arquitectónico es **conocimiento tácito o implícito**: reside únicamente en la cabeza de los arquitectos senior que lideraron la creación del sistema.

Hacer explícito este conocimiento permite que las decisiones sean **reutilizables, transferibles y reproducibles**, cubriendo de forma definitiva el hueco (*gap*) histórico entre los requisitos del cliente y la implementación del software.

| Dimensión | Conocimiento Tácito (No documentado) | Conocimiento Explícito (Capturado en AKM) |
|---|---|---|
| **Ubicación** | Mentes individuales de los ingenieros. | Repositorio Git (ADRs), wikis o herramientas AKM. |
| **Riesgo de Rotación** | **Crítico.** Si el arquitecto abandona la empresa, el conocimiento se pierde (*Bus Factor = 1*). | **Nulo.** El conocimiento permanece accesible para todo el equipo. |
| **Onboarding** | Lento, dependiente de reuniones orales repetitivas y confusas. | Rápido y autónomo; los nuevos miembros leen el histórico de decisiones. |
| **Evolución y Deuda** | Se introducen cambios incompatibles que degradan la arquitectura (*Erosión*). | Cada nuevo cambio se contrasta con las decisiones previas y sus restricciones. |

Tabla 3.1: Impacto organizativo del conocimiento tácito frente al explícito.

### 3.1.3 El Estándar Internacional ISO/IEC/IEEE 42010:2022

El estándar internacional para la descripción de la arquitectura de sistemas intensivos en software (**ISO/IEC/IEEE 42010:2022**) consolida esta visión al elevar formalmente las decisiones de diseño a elementos fundamentales:

**► Concepto Clave: Decisiones en el Estándar ISO/IEC/IEEE 42010**

El estándar estipula que toda descripción arquitectónica debe incorporar formalmente:
- Las **Decisiones de Diseño Arquitectónico** que modelan la solución.
- La **Justificación (*Rationale*)** técnica, funcional y de negocio que soporta cada decisión frente a sus alternativas.
- El registro de los **compromisos (*trade-offs*)** asumidos y los riesgos derivados.

## 3.2 Caracterización de Decisiones y Modelos de Captura

Dado que las decisiones de diseño se formulan originalmente en lenguaje natural humano, es imprescindible caracterizarlas mediante esquemas estructurados para evitar ambigüedades.

### 3.2.1 Aproximaciones de Captura de Decisiones

Existen tres aproximaciones principales para registrar y caracterizar las ADDs:

1. **Plantillas de Texto Estructurado y Ficheros Markdown (ADRs):** Esquemas estandarizados en texto plano versionados junto con el código fuente en Git. Es la aproximación dominante en la industria moderna por su ligereza y nula fricción.
2. **Ontologías Semánticas:** Modelos basados en grafos formales (RDF/OWL) que describen tipos de decisiones, estados, exclusiones y relaciones de compatibilidad procesables por motores de inferencia.
3. **Anotaciones en Código Fuente:** Metadatos incrustados directamente sobre clases o módulos (e.g., `@ArchDecision(.ªD-004", rationale="...")`).

### 3.2.2 El Modelo Canónico de Tyree y Akerman (2005)

Publicado por Jeff Tyree y Art Akerman (arquitectos de Capital One) en *IEEE Software*, definió la primera plantilla exhaustiva de la industria, estructurada en 14 atributos fundamentales:

| Atributo | Nombre en Inglés | Descripción Técnica |
|---|---|---|
| **Identificador** | `Name / ID` | Nombre breve y unívoco que sintetiza el problema resuelto (e.g., `AD-012: Cache Distribuida`). |
| **Problema de Diseño** | `Issue` | El reto arquitectónico o necesidad técnica precisa que se debe resolver. |
| **Decisión** | `Decision` | La solución adoptada y su alcance en el diseño. |
| **Estado** | `Status` | Ciclo de vida: *Pendiente, Decidida, Aprobada, Rechazada, Obsoleta.* |
| **Restricciones** | `Constraints` | Límites impuestos por el entorno, presupuesto, leyes (GDPR) o tecnología existente. |
| **Alternativas** | `Positions / Alternatives` | Soluciones tecnológicas o patrones viables considerados durante el análisis. |
| **Justificación** | `Rationale / Argument` | El razonamiento detallado de por qué la opción elegida supera a las alternativas. |
| **Implicaciones** | `Implications` | Consecuencias colaterales: nuevos componentes requeridos, costes de licencia, formación. |
| **Relaciones** | `Related Decisions` | Vínculos con otras decisiones (si la permite, requiere, excluye o modifica). |
| **Trazabilidad** | `Related Requirements` | Requisitos funcionales o atributos de calidad (RNF) que motivaron la decisión. |

Tabla 3.2: Atributos esenciales de la plantilla de Tyree y Akerman (2005).

**▲ ¡Atención / Ingeniería!: Flexibilidad en la Captura**

Uno de los mayores motivos de fracaso en la adopción de plantillas densas como la de Tyree & Akerman en empresas ágiles es la **fatiga burocrática**: obligar a los ingenieros a rellenar 14 campos densos para cada pequeña decisión paraliza el desarrollo. Por ello, las metodologías modernas definen un conjunto **mínimo obligatorio** (Problema, Decisión, Justificación, Consecuencias) y dejan los atributos adicionales como opcionales según la criticidad del subsistema.

## 3.3 La Vista de la Decisión y Procesos de AKM

### 3.3.1 La Vista de la Decisión en la Arquitectura

En el modelo de vistas 4+1 de Kruchten, cada vista se centra en un aspecto estructural o de ejecución. Sin embargo, ninguna de ellas respondía al *porqué* global.

**▶ Concepto Clave: Vista de la Decisión (*Decision View*)**

La **Vista de la Decisión** es una vista arquitectónica transversal que documenta las razones, alternativas, compromisos y justificaciones que motivan los elementos presentes en todas las restantes vistas del sistema.

### 3.3.2 Procesos del Ciclo de Vida de AKM

La Gestión del Conocimiento Arquitectónico (*Architecture Knowledge Management - AKM*) abarca cuatro grandes procesos interconectados:

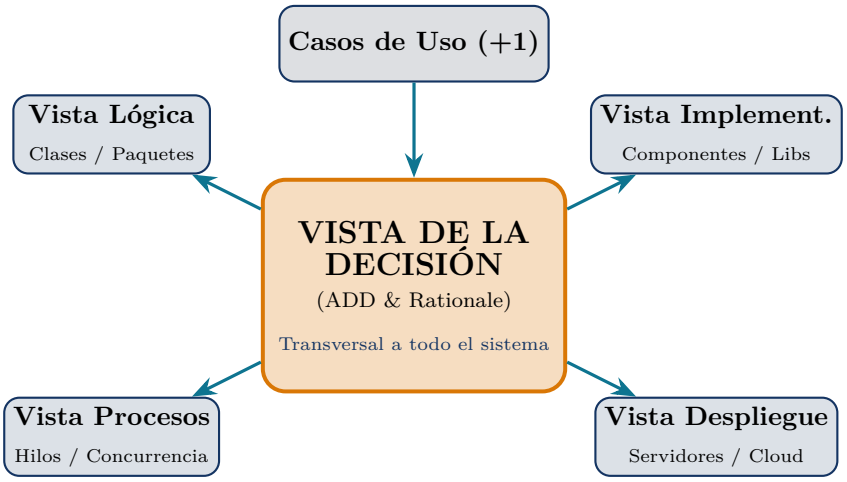



Figura 3.2: La Vista de la Decisión como dimensión transversal que ilumina las vistas del modelo 4+1.

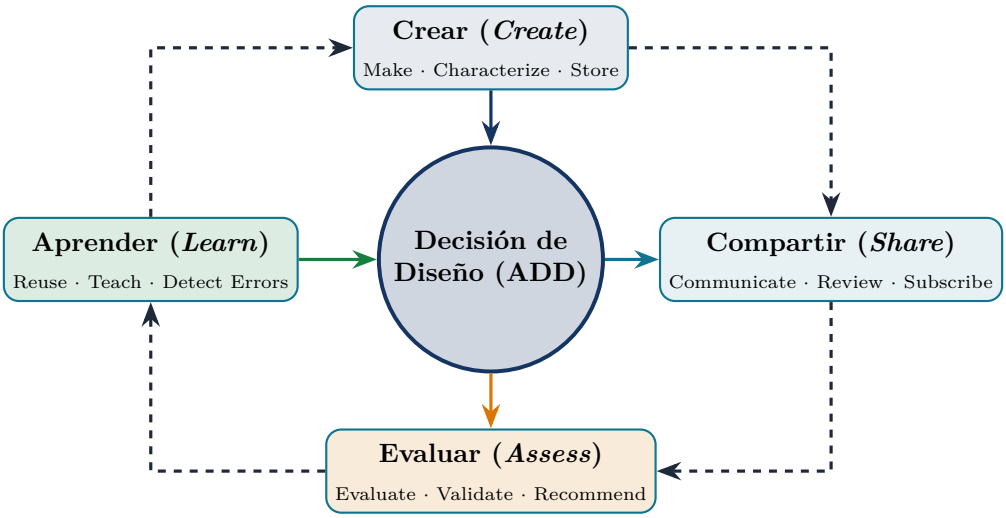



Figura 3.3: Ciclo de vida integral de los procesos de Architecture Knowledge Management (AKM).

### 3.3.3 Trazabilidad Externa: Forward y Backward Traceability

Las decisiones de diseño actúan como el nodo central que conecta los requisitos con los artefactos de software:

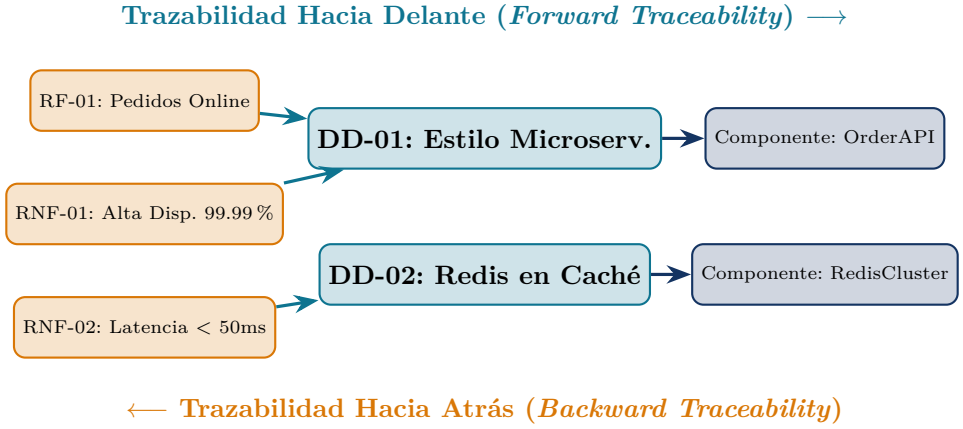



Figura 3.4: Trazabilidad Externa: Análisis de impacto hacia delante y verificación hacia atrás.

▶ **Concepto Clave: Modalidades de Trazabilidad Externa**

- **Trazabilidad Hacia Delante (*Forward Traceability*):** Del Requisito → a la Decisión → a la Arquitectura. Se utiliza para el **análisis de impacto**: si el cliente cambia o añade un requisito, el arquitecto identifica inmediatamente qué decisiones quedan invalidadas y qué componentes específicos deben refactorizarse.
- **Trazabilidad Hacia Atrás (*Backward Traceability*):** De la Arquitectura → a la Decisión → al Requisito. Se utiliza para **auditoría y justificación**: permite descubrir el origen legítimo de cualquier componente en el sistema o detectar código huérfano / decisiones erróneas que no responden a ningún requisito real.

### 3.3.4 Trazabilidad Interna y Relaciones Ontológicas entre Decisiones

Las decisiones de diseño no son átomos aislados: interactúan entre sí formando un grafo de dependencias internas. En modelos ontológicos se definen cuatro relaciones cardinales:

| Relación Ontológica | Semántica Formal | Ejemplo Real en Arquitectura |
|---|---|---|
| **Relacionada con** | Asociación informativa general. | La decisión de autenticación JWT está relacionada con el Gateway. |
| **Permite / Requiere** | $A$ habilita la adopción de $B$, o $B$ no puede existir sin $A$. | Adoptar *Database-per-Service* requiere implementar el patrón *SAGA*. |
| **Excluye** | $A$ y $B$ son mutuamente excluyentes. | Elegir una arquitectura monolítica embebida de 2 capas **excluye** desplegar un clúster de microservicios con Kubernetes. |
| **En Conflicto con** | $A$ y $B$ entran en fricción de atributos de calidad. | Requerir cifrado de extremo a extremo entra en conflicto con inspección de paquetes en el API Gateway. |

Tabla 3.3: Tipología de relaciones en la trazabilidad interna entre decisiones.

**[X] Anti-Patrón de Diseño: Explosión Combinatoria por Granularidad Excesiva**

Si el equipo intenta documentar decisiones a un nivel de granularidad microscópico (e.g., cada nombre de variable o cada bucle `for`), el grafo de dependencias se vuelve inabarcable e inútil. Como regla de oro, **las decisiones arquitectónicas deben acotarse al nivel de componentes, paquetes, estilos, tecnologías clave e interfaces principales**.

## 3.4 Herramientas de AKM y su Evolución Histórica

Desde 2004, la comunidad científica e industrial desarrolló herramientas para capturar, almacenar y razonar sobre el conocimiento arquitectónico:

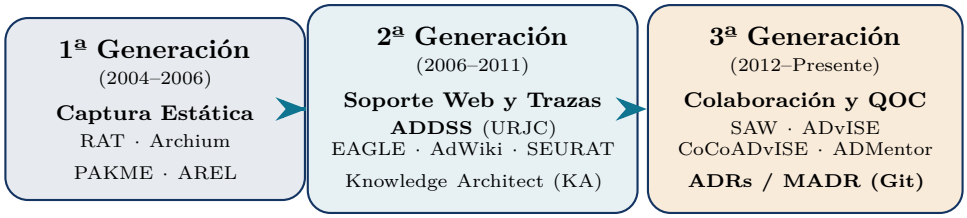



Figura 3.5: Cronología evolutiva de las tres generaciones de herramientas de AKM.

### 3.4.1 Herramientas Destacadas

- **PAKME (2006):** Herramienta web basada en la plantilla de Tyree & Akerman con soporte para atributos de calidad y patrones de diseño (Babar et al.).
- **Archium:** Herramienta de modelado y extensión de lenguaje que emitía alertas cuando se introducían decisiones incompatibles.
- **ADDSS (Architecture Design Decision Support System, 2006–2011):** Desarrollada en la Universidad Rey Juan Carlos (Capilla et al.). Fue pionera en:
  - Gestión web de decisiones asociadas a vistas arquitectónicas.
  - Enlaces bidireccionales de trazabilidad entre requisitos, decisiones y evolución de las arquitecturas a lo largo de sucesivas iteraciones.
  - Generación automática de documentación de arquitectura en PDF.
- **SAW (Software Architecture Warehouse - Nowak, Pautasso):** Formato Wiki colaborativo para captura de decisiones en arquitecturas SOA/Cloud con búsqueda facetada por tags y modelos fuzzy de consenso.
- **ADvISE y CoCoADvISE:** Utilizan el método **QOC (*Question-Option-Criteria*)** para representar el espacio del problema mediante preguntas, opciones de diseño evaluadas frente a criterios de calidad (RNF) y dependencias entre opciones.
- **ADMentor (Olaf Zimmermann et al.):** Extensión sobre Enterprise Architect que asiste a los ingenieros en el modelado de decisiones para arquitecturas Cloud en colaboración con grandes corporaciones como ABB.

## 3.5 El Estándar Industrial MADR (Markdown Architectural Decision Records)

En el desarrollo de software moderno con metodologías ágiles y DevOps, la documentación pesada fuera de los repositorios tiende a desactualizarse rápidamente. La industria ha convergido de forma unánime hacia los **Architecture Decision Records (ADR)** versionados en Git junto al código, siendo el formato **MADR** (*Markdown Architectural Decision Records*) el estándar más adoptado.

### 3.5.1 Estructura Oficial de una Plantilla MADR

```
# [Identificador y Titulo: Solucion y Problema resuelto]

* Status: [proposed | rejected | accepted | deprecated | superseded by ADR-0005]
* Date: [YYYY-MM-DD]
* Deciders: [Lista de arquitectos / ingenieros participantes]

## Context and Problem Statement
[Descripcion concisa del contexto del sistema y el problema tecnico o funcional que exige
     tomar una decision]

## Decision Drivers
* [RF-01 o Atributo de Calidad: p. ej., Latencia < 50ms bajo 10.000 req/s]
* [Coste de infraestructura mensual acotado a 500$]
* [Cumplimiento estricto con la normativa de privacidad GDPR]

## Considered Options
* [Opcion 1: Nombre de la tecnologia, estilo o patron]
* [Opcion 2: Alternativa tecnologica 2]
* [Opcion 3: Alternativa tecnologica 3]

## Decision Outcome
Chosen option: "[Opcion 1]", because [justificacion concluyente que resuelve las fuerzas
    de los decision drivers y supera a las demas alternativas].

### Positive Consequences
* [Consecuencia positiva 1 en el contexto del proyecto]
* [Consecuencia positiva 2]

### Negative Consequences
* [Compromiso / Desventaja asumida (Trade-off)]
* [Riesgo tecnico o coste operativo derivado]

## Pros and Cons of the Options

### [Opcion 1]
* Good, because [argumento positivo contextualizado]
* Bad, because [argumento negativo contextualizado]

### [Opcion 2]
* Good, because [argumento positivo contextualizado]
* Bad, because [argumento negativo contextualizado]
```

Listing 3.1: Estructura formal de una plantilla MADR v3.0

### 3.5.2 Errores Clave a Evitar en MADR (Examen y Práctica Profesional)

En exámenes universitarios y revisiones de diseño en empresas, los ingenieros noveles suelen cometer errores graves al rellenar plantillas MADR:

**[X] Anti-Patrón de Diseño: Errores Críticos en el Uso de MADR**

1. **Reutilizar requisitos funcionales como *Decision Drivers* indiscriminadamente:** Los requisitos de dominio y negocio deben usarse con precisión; un requisito solo debe ser el motor de la decisión que directamente lo satisface.

2. **Proponer opciones consideradas absurdas o inconexas:** Si el problema es elegir un mecanismo de comunicación asíncrona entre microservicios, no tiene sentido poner como opciones: *1. RabbitMQ, 2. Estilo por Capas, 3. Singleton.* Las alternativas deben pertenecer al mismo nivel de abstracción y espacio de la solución.
3. **Argumentos genéricos vacíos de contexto:**
   - **Inaceptable:** `Good, because es escalable` (no dice nada).
   - **Correcto:** `Good, because permite escalar horizontalmente los consumidores del topic de pedidos de 2 a 16 réplicas en Kubernetes mediante KEDA durante el Black Friday sin pérdida de mensajes.`
4. **Omitir las consecuencias negativas:** Toda decisión arquitectónica implica compromisos (*trade-offs*). Un ADR sin desventajas revela que el arquitecto no ha analizado los costes ocultos o la complejidad operativa de su elección.

## 3.6 Pensamiento Crítico y Reflexión en la Arquitectura: Modelo Mind 1 / Mind 2

La toma de decisiones de diseño está frecuentemente contaminada por **sesgos cognitivos**:

- **Sesgo de la Ley del Instrumento de Maslow:** *«Si tu única herramienta es un martillo, todos los problemas te parecerán clavos»* (e.g., el desarrollador que insiste en usar siempre microservicios o el patrón Mediator para cualquier aplicación sencilla).
- **Sesgo de Novedad (*Hype-Driven Development*):** Elegir la última librería o tecnología de moda aparecida en Twitter/GitHub sin analizar si encaja con las restricciones reales del problema.

Para neutralizar estos sesgos, la disciplina incorpora el **pensamiento reflectivo y crítico** mediante el modelo **Mind 1 / Mind 2** (inspirado en la teoría de procesos duales del premio Nobel Daniel Kahneman):

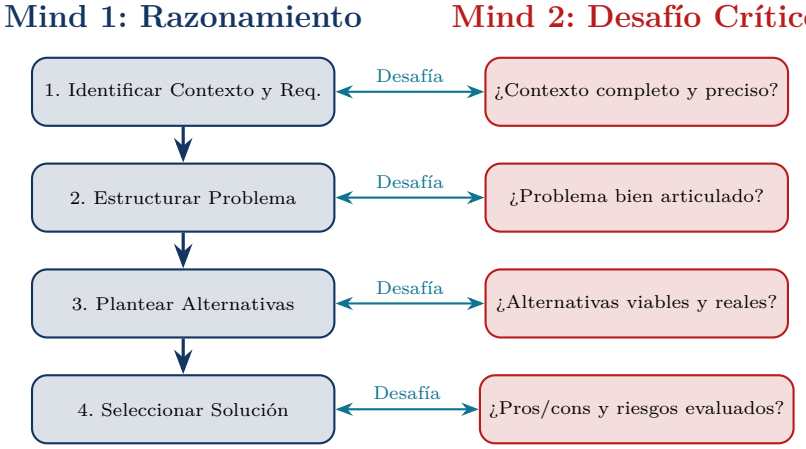



Figura 3.6: El modelo de pensamiento crítico Mind 1 / Mind 2 en el diseño arquitectónico.

**▶ Concepto Clave: Roles en el Modelo Mind 1 / Mind 2**

- **Mind 1 (Design Reasoning - El Creador):** Es el ingeniero o equipo que formula el problema, analiza los requisitos, genera alternativas y propone una solución técnica lógica.
- **Mind 2 (Reflective Thinking - El Abogado del Diablo):** Es el revisor o arquitecto crítico que pone en duda metódica cada asunción de Mind 1:
  - *¿Se están resolviendo los requisitos correctos?*

- *¿Se descartaron alternativas viables por sesgo personal?*
- *¿Qué pasará si el tráfico se multiplica por 10 o la red se particiona?*

La interacción dialéctica entre Mind 1 y Mind 2 eleva radicalmente la calidad técnica de las decisiones y previene desastres operativos antes de escribir la primera línea de código.

## 3.7 Ejercicios Resueltos y Casos Prácticos de Examen

### ★ Ejercicio Resuelto: Redacción Formal de un Registro MADR para Migración Monolito a Microservicios

**Enunciado:**
Una empresa de distribución alimentaria cuenta con un monolito legado con 6 módulos (*Clientes, Pedidos, Reparto y Rutas, Estadísticas, Incidencias y Pagos*) y dos bases de datos SQL sobrecargadas. Se planea exponer el sistema hacia aplicaciones móviles y colaboradores externos mediante APIs HTTP/REST seguras. Redacte el registro arquitectónico MADR formal que capture la decisión del componente de acceso unificado.

**Solución Razonada:**

```
# ADR-001: Implementacion de API Gateway para Acceso a Microservicios

* Status: accepted
* Date: 2026-11-09
* Deciders: Arquitecto Jefe, Equipo de Plataforma

## Context and Problem Statement
El sistema monolítico actual expone directamente puertos de base de datos a
    clientes web y de escritorio internos. Con la migración a microservicios y la
    entrada de apps móviles, se requiere un punto de entrada único que aísle a los
     clientes de la topología interna de red y centralice la autenticación,
    limitación de tasa y enrutamiento.

## Decision Drivers
* RNF-Seguridad: Centralizar la validación de tokens OAuth2/JWT antes de entrar a
    la red interna.
* RNF-Rendimiento: Enrutamiento inverso con latencia añadida inferior a 5 ms.
* RNF-Mantenibilidad: Evitar que los clientes móviles conozcan las IPs de los 6
    microservicios.

## Considered Options
* 0001-Kong API Gateway (Basado en NGINX/OpenResty)
* 0002-Spring Cloud Gateway (Basado en Netty reactivo)
* 0003-Llamadas directas cliente-microservicio (Sin Gateway)

## Decision Outcome
Chosen option: "Kong API Gateway", because ofrece rendimiento óptimo en C/Lua,
    plugins estándar listos para producción (Rate Limiting, JWT, CORS) y soporte
    nativo para despliegues en Kubernetes.

### Positive Consequences
* Punto de control perimetral único para auditoría y seguridad.
* Los microservicios internos no tienen que reimplementar validación de tokens ni
    políticas CORS.

### Negative Consequences
* Introduce un componente intermedio que debe desplegarse en alta disponibilidad
    para evitar ser un punto único de fallo (SPOF).

## Pros and Cons of the Options
```

```
### Kong API Gateway
* Good, because latencia ultrabaja (<2ms) y bajo consumo de memoria.
* Good, because soporte para plugins de seguridad y observabilidad Prometheus.
* Bad, because la configuración avanzada de plugins personalizados exige
    conocimientos de Lua.

### Spring Cloud Gateway
* Good, because integración directa con el stack Java de la empresa.
* Bad, because mayor consumo de memoria JVM y rendimiento inferior bajo picos de
    50.000 peticiones/s.

### Llamadas directas (Sin Gateway)
* Bad, because acopla a las aplicaciones cliente a la red interna y multiplica el
    riesgo de vulnerabilidades de seguridad en cada microservicio.
```

### ★ Ejercicio Resuelto: Análisis de Impacto mediante Trazabilidad Bidireccional

**Enunciado:**
Durante el despliegue del sistema alimentario, el departamento legal informa de que, debido a la normativa de protección de datos, ningún dato bancario de clientes puede residir en servidores situados fuera de la Unión Europea.

1. Utilice la trazabilidad hacia delante (*Forward Traceability*) para determinar qué decisiones y componentes se ven impactados.
2. Indique qué relación ontológica surge entre la decisión de almacenamiento bancario y la decisión de infraestructura en la nube.

**Solución Razonada:**

1. **Análisis Forward:**

   Requisito Legal GDPR ⟶ ADD-004 (Almacén de Pagos) ⟶ Microservicio de Pagos y DB PostgreSQL

   El análisis identifica que la decisión `ADD-004` queda invalidada si apuntaba a una región `us-east-1` de AWS. El componente `PaymentService` y su base de datos son los únicos que deben reubicarse en la región `eu-central-1` (Frankfurt) sin necesidad de modificar los otros 5 microservicios (Rutas, Estadísticas, etc.), minimizando el coste de refactorización.
2. **Relación Ontológica:** La decisión de cumplimiento legal **excluye** la opción de utilizar proveedores de nube con almacenamiento exclusivo fuera de la UE y **requiere** el cifrado en reposo con claves gestionadas localmente (KMS europeo).

## 3.8 Resumen Ejecutivo y Preguntas de Autoevaluación

**Resumen de Conceptos Clave del Tema 3**

- **Arquitectura = Decisiones (Jan Bosch):** La arquitectura es el resultado acumulado de decisiones de diseño justificadas; los componentes y conectores son la consecuencia de dichas decisiones.
- **Evaporación del Conocimiento:** Si el conocimiento tácito no se explicita, se pierde al rotar el equipo, provocando erosión arquitectónica.
- **ISO/IEC/IEEE 42010:2022:** Reconoce formalmente las decisiones y su justificación (*rationale*) como elementos arquitectónicos de primer orden.
- **Vista de la Decisión:** Dimensión transversal a las 4+1 vistas tradicionales y a los requisitos.
- **Trazabilidad Bidireccional:** *Forward* (del requisito a la arquitectura para análisis de impacto) y *Backward* (de la arquitectura al requisito para auditoría y depuración de causas).
- **MADR en Git:** Estándar ligero y versionable para documentar decisiones en Markdown. Exige alternativas viables, justificación con *because* y consecuencias contextualizadas.
- **Modelo Mind 1 / Mind 2:** Diálogo crítico entre el razonamiento creador (Mind 1) y el pensamiento reflexivo desafiante (Mind 2) para erradicar sesgos cognitivos.

### 3.8.1 Batería de Preguntas de Autoevaluación

1. **¿Por qué afirma Jan Bosch que documentar únicamente componentes y conectores es insuficiente para describir una arquitectura software?**
   *Respuesta orientativa:* Porque los diagramas solo muestran el estado final de la estructura, pero ocultan los motivos subyacentes, los atributos de calidad que la motivaron y las alternativas tecnológicas descartadas. Sin este conocimiento, los futuros mantenedores violarán las asunciones de diseño originales.

2. **¿Cuál es la diferencia entre trazabilidad forward y backward en el marco de las ADDs?**
   *Respuesta orientativa:* La trazabilidad forward viaja del requisito hacia la decisión y la arquitectura para estimar el impacto de un cambio en los requisitos. La trazabilidad backward viaja de la arquitectura hacia la decisión y el requisito para justificar la existencia de un componente o rastrear el origen de una decisión errónea.

3. **¿Qué peligro entraña una red de trazabilidad interna con granularidad demasiado fina?**
   *Respuesta orientativa:* Produce una explosión combinatoria de dependencias que eleva desproporcionadamente la complejidad del mantenimiento de la documentación, provocando que los ingenieros abandonen la captura de conocimiento.

4. **En el formato MADR, ¿por qué es inaceptable justificar una alternativa con la frase "Good, because es escalable"?**
   *Respuesta orientativa:* Porque es un argumento abstracto y genérico desprovisto del contexto del problema. La justificación debe explicar de qué forma precisa y bajo qué métricas la alternativa satisface las restricciones y requisitos específicos del sistema.

5. **¿Cómo opera el modelo Mind 1 / Mind 2 para prevenir el sesgo del instrumento de Maslow en la elección de patrones de diseño?**
   *Respuesta orientativa:* Mind 1 propone la solución guiado por la intuición y la experiencia previa, mientras que Mind 2 actúa como evaluador crítico que pone en duda si la solución elegida responde a una necesidad real del problema o si simplemente se propuso por afinidad personal del desarrollador.

# CAPÍTULO 4

# Evaluación de Arquitecturas Software y Método ATAM

**Objetivos de Aprendizaje del Tema 4**

Al finalizar este capítulo, el estudiante e ingeniero de software será capaz de:

- Comprender por qué los **Requisitos No Funcionales (RNF)** o **Atributos de Calidad (Quality Attributes - QAs)** son los que verdaderamente moldean y determinan la **forma final** de una arquitectura software.
- Contrastar formalmente la cardinalidad y el alcance de los Requisitos Funcionales (RF) frente a los Atributos de Calidad (RNF).
- Cuantificar y operacionalizar atributos de calidad vagos en **valores numéricos medibles** y verificables mediante pruebas empíricas.
- Dominar la taxonomía de atributos de calidad en tiempo de ejecución (*Runtime*), fuera de ejecución (*Non-Runtime*) y de negocio, así como los estándares internacionales **ISO/IEC 25010:2011** y **ISO/IEC 25010:2023**.
- Diseñar **Escenarios de Calidad** formales según el estándar del SEI, desglosando con precisión sus seis partes canónicas: *Fuente, Estímulo, Entorno, Artefacto, Respuesta y Medida de la Respuesta.*
- Construir y priorizar un **Árbol de Utilidad (*Utility Tree*)** utilizando la matriz bidireccional de prioridad $(Importancia, Dificultad) \in \{H, M, L\}^2$.
- Comprender el panorama de métodos de evaluación de arquitectura (*SAAM, ARID, CBAM*) y dominar el método de referencia de la industria: **ATAM (*Architecture Tradeoff Analysis Method*)**.
- Identificar, clasificar y documentar los cuatro hallazgos cardinales de una evaluación arquitectónica: **Riesgos**, **No-Riesgos**, **Puntos de Sensibilidad (*Sensitivity Points*)** y **Compromisos (*Trade-offs*)**.
- Ejecutar las dos fases y los nueve pasos secuenciales de ATAM, comprendiendo los roles del equipo evaluador y la interacción con los *stakeholders*.

## 4.1 El Rol Crítico de los Atributos de Calidad en la Arquitectura

### 4.1.1 Los Requisitos No Funcionales Determinan la Forma del Software

Un error común en ingeniería de software novata consiste en diseñar un sistema basándose únicamente en sus casos de uso funcionales (e.g., “el sistema debe permitir a un usuario autenticarse y realizar un pedido”).

Si la arquitectura solo respondiera a requerimientos funcionales, cualquier estructura de código que ejecutara la lógica de negocio sería válida, incluyendo un monolito desordenado de 50,000 líneas en un único archivo.

**★ Principio Fundamental de Diseño: Los RNF Determinan la Forma de la Arquitectura**

Las arquitecturas software **no son sólo el resultado de responder a Requisitos Funcionales (RF)**; son los **Requisitos No Funcionales (RNF) o Atributos de Calidad (*Quality Attributes - QAs*)** los que dictan de forma determinante la **forma, topología y organización final** de la arquitectura.

| Dimensión de Comparación | Requisito Funcional (RF) | Requisito No Funcional / QA (RNF) |
|---|---|---|
| **¿Qué describe?** | Lo que el sistema **hace** (capacidades, casos de uso, transformaciones de datos). | Cómo de bien el sistema se comporta mientras lo hace (atributos de calidad). |
| **Cardinalidad de Uso** | Se utiliza **una sola vez** en el proceso de diseño. | Puede ser usado **múltiples veces** a lo largo del proceso de diseño. |
| **Alcance en la Arquitectura** | Afecta a una **parte concreta y localizada** (un método, clase o controlador específico). | Es transversal: **afecta a diversas partes y subsistemas** simultáneamente. |
| **Facilidad de Verificación** | Binaria: la función calcula el resultado correcto o falla (pasa/falla un test unitario). | Compleja y cuantitativa: exige métricas de latencia, disponibilidad, carga y observabilidad. |

Tabla 4.1: Diferencias estructurales entre Requisitos Funcionales y Atributos de Calidad.

### 4.1.2 La Dificultad Intrínseca de Evaluar Diseños

Evaluar un sistema ya en ejecución (*benchmarking*, pruebas de estrés) es un problema técnico habitual. Sin embargo, **evaluar una arquitectura software antes de su construcción** entraña una dificultad extrema:

- **Evaluamos un diseño abstracto sobre papel o modelo, no un programa compilado:** No podemos ejecutar un *profiler* de CPU sobre un diagrama UML de paquetes.
- **Existen múltiples soluciones candidatas en competencia:** ¿Conviene una base de datos relacional particionada o un clúster NoSQL orientado a documentos? ¿Conviene comunicación síncrona gRPC o un bus asíncrono RabbitMQ?
- **Imposibilidad de maximizar todos los atributos a la vez:** El cliente desearía un sistema infinitamente escalable, con latencia cero, seguridad militar impenetrable y coste económico nulo. La realidad física y matemática obliga a gestionar compromisos (*trade-offs*).

### 4.1.3 Operacionalización Cuantitativa: De Deseos Vagos a Métricas Medibles

Para que un atributo de calidad pueda evaluarse en una arquitectura, es estrictamente obligatorio transformarlo en un **valor numérico medible o en un rango cuantitativo estricto**. Si un requerimiento no es cuantificable, no puede evaluarse ni auditarse formalmente.

**[X] Anti-Patrón de Diseño: Especificación Vaga de Calidad**

Formulaciones ambiguas como:

- **Inaceptable:** `«Se desea que la encriptación de datos sea altamente segura»`.
- **Inaceptable:** `«El sistema debe ser muy rápido y escalable»`.

Estas afirmaciones son subjetivas y no permiten al arquitecto elegir entre algoritmos ni verificar el

cumplimiento del diseño.

✓ **Ejemplo Práctico: Operacionalización Cuantitativa Rigurosa**

- **Formulación Correcta (Cifrado y Rendimiento):** *«Al cifrar un volumen de* 1 *GB de datos mediante el algoritmo AES con clave de* 256 *bits en modo GCM sobre un hilo de CPU estándar, el tiempo de procesamiento no debe superar los* 6 *segundos, manteniendo una tasa de uso de CPU inferior al* 35 %*».*
- **Formulación Correcta (Latencia de Consulta):** *«Bajo una carga concurrente de* 1,000 *peticiones por segundo, el percentil 99 (p*99*) del tiempo de respuesta del servicio de autenticación debe ser inferior a* 30 *ms».*

## 4.2 Taxonomías de Atributos de Calidad y Estándar ISO/IEC 25010

### 4.2.1 Clasificación de Atributos: Runtime, Non-Runtime y Negocio

Los atributos de calidad se estructuran habitualmente en tres grandes categorías operacionales:

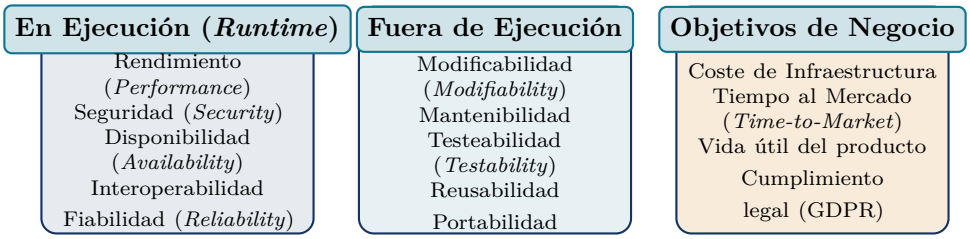



Figura 4.1: Taxonomía operacional de los Atributos de Calidad en arquitectura software.

Los atributos se descomponen jerárquicamente en **sub-atributos** de granularidad más fina para facilitar su medición directa:

$$\textbf{Mantenibilidad} \longrightarrow \begin{cases} \text{Complejidad ciclomática (sub-QA nivel 1)} \\ \text{Modularidad y acoplamiento (sub-QA nivel 1)} \\ \text{Analizabilidad y facilidad de prueba (sub-QA nivel 1)} \end{cases}$$

### 4.2.2 El Modelo de Calidad ISO/IEC 25010:2011 frente a ISO/IEC 25010:2023

El estándar internacional **ISO/IEC 25010** (parte de la familia SQuaRE – *Software Product Quality Requirements and Evaluation*) representa la referencia formal de la industria para evaluar la calidad del software:

▲ **¡Atención / Ingeniería!: Novedades Fundamentales de la Edición ISO/IEC 25010:2023**

En la revisión publicada en 2023, la norma incorporó ajustes esenciales para los sistemas modernos:

1. **Inclusión formal de *Safety* (Seguridad Funcional y Operativa):** Capacidad del sistema para operar en un estado que previene daños o amenazas a la vida humana, la salud de las personas, las propiedades materiales o el medio ambiente (vital en automoción, robótica y electromedicina).
2. **Evolución terminológica:** Se refina *Usability* hacia ***Interaction Capability*** (abarcando interfaces por voz, multimodales e IA) y *Portability* hacia ***Flexibility***.

| Característica ISO 25010 | Sub-características Clave | Definición de Ingeniería |
|---|---|---|
| **Adecuación Funcional** | Completitud, corrección y pertinencia. | Capacidad del software de proveer funciones que satisfacen las necesidades declaradas. |
| **Eficiencia de Desempeño** | Comportamiento temporal, uso de recursos, capacidad. | Relación entre el nivel de rendimiento y la cantidad de recursos de cómputo utilizados. |
| **Compatibilidad** | Coexistencia e interoperabilidad. | Capacidad de intercambiar información con otros sistemas compartiendo el mismo entorno. |
| **Usabilidad / Capacidad Interac.** | Aprendizaje, operabilidad, protección de errores, estética. | Facilidad con la que los usuarios pueden utilizar el sistema para lograr sus metas con eficacia. |
| **Fiabilidad (*Reliability*)** | Madurez, disponibilidad, tolerancia a fallos, recuperación. | Capacidad de mantener un nivel de rendimiento especificado en un periodo de tiempo. |
| **Seguridad (*Security*)** | Confidencialidad, integridad, no repudio, autenticidad. | Protección de información y datos para impedir accesos o modificaciones no autorizadas. |
| **Mantenibilidad** | Modularidad, reusabilidad, analizabilidad, modificabilidad, testeabilidad. | Facilidad con la que el software puede ser modificado para corregir fallos o añadir mejoras. |
| **Flexibilidad / Portabilidad** | Adaptabilidad, facilidad de instalación, reemplazabilidad. | Facilidad para trasladar el software de un entorno hardware o nube a otro. |

Tabla 4.2: Características de calidad del producto software según el modelo ISO/IEC 25010.

## 4.3 Escenarios de Calidad: Cuantificación Medible de Atributos

Los atributos de calidad son conceptos abstractos. Para poder utilizarlos en una evaluación técnica, deben instanciarse en **Escenarios de Calidad**.

> ▶ **Concepto Clave: Escenario de Calidad**
>
> Un **Escenario de Calidad** es una descripción breve y formal de una situación concreta en la que un estímulo externo o interno interactúa con el sistema, especificando las condiciones del entorno y una medida objetiva y verificable de la respuesta esperada.

Existen dos tipos de escenarios según su grado de abstracción:

- **Escenarios Generales:** Independientes del dominio del sistema (e.g., "ante un fallo en un nodo de base de datos, el sistema se recupera sin pérdida de transacciones").
- **Escenarios Concretos:** Especificados con parámetros numéricos exactos para un sistema particular (e.g., "si falla el nodo primario de MongoDB en el cluster de pedidos durante la campaña de Navidad, el replica set debe promover un secundario en menos de 4 segundos").

### 4.3.1 Las 6 Partes Canónicas de un Escenario (Estándar SEI)

El *Software Engineering Institute* (SEI) de la Universidad Carnegie Mellon estandarizó las seis partes que componen inequívocamente todo escenario de calidad profesional:




Figura 4.2: Las seis partes canónicas de un Escenario de Calidad según el SEI.

1. **Fuente del Estímulo (*Source of Stimulus*):** La entidad (humana o automatizada) que origina la acción (un usuario legítimo, un hacker malicioso, un sensor IoT, el planificador del sistema operativo).
2. **Estímulo (*Stimulus*):** La condición o evento que arriba al sistema y que exige una respuesta (una transacción de compra, un fallo de conexión a red, una petición de cambio en el código fuente).
3. **Entorno (*Environment*):** El estado operacional en el que se encuentra el sistema cuando arriba el estímulo (modo normal, pico de tráfico máximo, modo degradado, tiempo de desarrollo/diseño).
4. **Artefacto Afectado (*Artifact*):** La parte de la arquitectura o subsistema que recibe el estímulo (el servicio de autenticación, el cluster de Kafka, el esquema relacional de la base de datos).
5. **Respuesta (*Response*):** La acción o actividad que el sistema o los ingenieros ejecutan en consecuencia (procesar la compra, levantar un contenedor sustituto, aislar el fallo, realizar la refactorización sin efectos colaterales).
6. **Medida de la Respuesta (*Response Measure*):** El criterio cuantitativo y objetivo con el que se juzga si la respuesta satisface el requisito (latencia media inferior a 2 s, tiempo de recuperación menor a 5 minutos, tiempo de modificación en menos de 4 horas-persona).

### 4.3.2 Ejemplos de Escenarios Canónicos por Atributo

✓ **Ejemplo Práctico: Escenario de Rendimiento (*Performance*)**

*«Cuando 100 usuarios inician concurrentemente la transición 'completar pago' (Estímulo) desde el navegador web (Fuente), el componente de pagos (Artefacto), bajo condiciones operacionales normales (Entorno), procesará las solicitudes (Respuesta) con una latencia media inferior a 3 segundos (Medida de la respuesta)».*

✓ **Ejemplo Práctico: Escenario de Disponibilidad (*Availability*)**

*«Un mensaje de red malformado e imprevisto (Estímulo) es transmitido por un sistema externo (Fuente) al proceso de ingesta de telemetría (Artefacto) en horario de producción continuo (Entorno). El sistema registra el error, informa al operador de guardia y continúa operando ininterrumpidamente (Respuesta) con cero segundos de tiempo de caída acumulado (Medida de la respuesta)».*

✓ **Ejemplo Práctico: Escenario de Modificabilidad (*Modifiability*)**

*«El equipo de producto desea cambiar la pasarela de pagos actual de Redsys a Stripe (Estímulo). El desarrollador (Fuente) introduce las modificaciones en la capa de adaptadores de infraestructura (Artefacto) en tiempo de diseño (Entorno). La modificación se completa sin efectos colaterales en las reglas de negocio (Respuesta) con un esfuerzo de implementación y pruebas inferior a 8 horas-persona (Medida de la respuesta)».*

## 4.4 El Árbol de Utilidad (*Utility Tree*)

El **Árbol de Utilidad** es la herramienta estructural que traduce las metas de alto nivel del negocio y los requerimientos no funcionales abstractos en un catálogo priorizado de escenarios concretos.

### 4.4.1 Estructura Jerárquica del Utility Tree

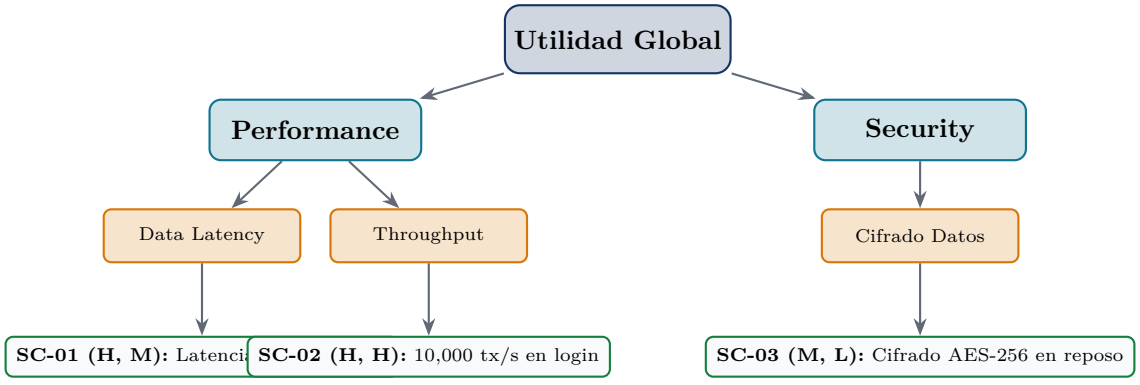



Figura 4.3: Desglose jerárquico de un Árbol de Utilidad (Utility Tree).

### 4.4.2 Matriz de Priorización de Escenarios $(Importancia, Dificultad)$

En un sistema real pueden definirse fácilmente entre 30 y 80 escenarios. Dado que los recursos y el tiempo del equipo son finitos, es imposible analizar todos los escenarios con el mismo rigor. El Utility Tree introduce una **priorización bidimensional** asignando un par de letras $(H, M, L) \in \{\text{High}, \text{Medium}, \text{Low}\}$:

$$\textbf{Prioridad} = (\textbf{Importancia para el Negocio}, \textbf{Dificultad Técnica de Implementación}) \quad (4.1)$$

| Categoría | Vector | Acción del Equipo de Evaluación (ATAM) |
|---|---|---|
| **Críticos** | **(H, H)** | **Máxima prioridad absoluta.** Escenarios cruciales para el negocio y de altísima complejidad técnica. Se analizan primero y en profundidad. |
| **Importantes** | **(H, M)** | **Alta prioridad.** Esenciales para el negocio y con dificultad media. |
| **Riesgo Técnico** | **(M, H)** | Interés moderado para el negocio pero técnicamente desafiantes. Requieren vigilancia. |
| **Secundarios** | **(M, M), (L, H)** | Se evalúan si sobra tiempo en el taller o mediante pruebas automatizadas secundarias. |
| **Baja Prioridad** | **(L, L), (M, L)** | No justifican un análisis profundo durante la evaluación arquitectónica. |

Tabla 4.3: Matriz de priorización de escenarios en el Utility Tree.

## 4.5 El Método ATAM (*Architecture Tradeoff Analysis Method*)

Creado por Rick Kazman, Mark Klein y Paul Clements en el Software Engineering Institute (SEI) de la Universidad Carnegie Mellon, el método **ATAM** es el estándar de referencia mundial para evaluar arquitecturas de software en sectores de alta exigencia (automoción, aeroespacial, financiero, defensa y telecomunicaciones).

### 4.5.1 Los Cuatro Hallazgos Cardinales de ATAM

El objetivo fundamental de ATAM no es emitir una calificación numérica del software, sino descubrir y documentar formalmente cuatro tipos de hallazgos en el diseño:

▶ **Concepto Clave: Hallazgos Cardinales de ATAM**

1. **Riesgo (*Risk*):** Una decisión arquitectónica que no ha sido suficientemente analizada o justificada y que puede comprometer gravemente la consecución de un atributo de calidad requerido.
2. **No-Riesgo (*Non-Risk*):** Una decisión arquitectónica sólida y bien fundamentada que garantiza que el atributo de calidad asociado se cumplirá holgadamente.
3. **Punto de Sensibilidad (*Sensitivity Point*):** Un componente, propiedad o parámetro de la arquitectura en el cual un cambio minúsculo produce una alteración desproporcionada en un atributo de calidad.
4. **Compromiso (*Trade-off*):** Una decisión de diseño que actúa simultáneamente sobre dos o más atributos de calidad contrapuestos, mejorando uno a costa de degradar otro.

✓ **Ejemplo Práctico: Ejemplos Reales de Hallazgos en ATAM**

- **Riesgo:** *«Las reglas para escribir módulos de lógica de negocio en la capa 2 del estilo de 3 capas no están articuladas formalmente. Esto podría provocar duplicación de código comprometiendo la modificabilidad de la capa 3».*
- **No-Riesgo:** *«Asumiendo una tasa de llegada de 1 mensaje por segundo y un tiempo de cómputo inferior a 30 ms, la existencia de un hilo prioritario garantiza que el plazo límite de 1 segundo se cumplirá con certeza matemática».*

- **Punto de Sensibilidad:** *«El rendimiento de todo el sistema de pedidos es altamente sensible a la fiabilidad del Message Broker: a partir de* 1,000 *mensajes/s, cualquier retardo en el cluster Kafka colapsa los servicios subordinados».*
- **Compromiso (*Trade-off*):** *«Si incrementamos la longitud de clave a AES-256 para maximizar la Seguridad en la codificación de vídeo en streaming, se reduce drásticamente el Rendimiento del microcontrolador o bien se dispara el Coste del hardware requerido».*

### 4.5.2 Roles y Actores en el Proceso ATAM

Un taller de evaluación ATAM requiere la participación coordinada de tres grupos:

| Rol en ATAM | Composición / Perfil | Responsabilidad Principal |
|---|---|---|
| **Clientes y Stakeholders** | Directores de negocio, usuarios finales, responsables de producto, operadores. | Presentar las metas comerciales, restricciones del mercado y definir la importancia de los escenarios. |
| **Arquitectos de Software** | Diseñadores técnicos del sistema, arquitectos de software, líderes técnicos. | Presentar la arquitectura propuesta, justificar las decisiones de diseño y responder a los escenarios planteados. |
| **Equipo Evaluador ATAM** | Especialistas externos e independientes: *Moderador, Redactor de escenarios, Controlador de tiempo, Monitor del proceso, Entrevistador.* | Facilitar las dinámicas, cuestionar asunciones implícitas, construir el Utility Tree y redactar el informe técnico final. |

Tabla 4.4: Roles y responsabilidades de los participantes en una evaluación ATAM.

### 4.5.3 Las Fases y los 9 Pasos Secuenciales de ATAM

El método ATAM se estructura en dos fases operacionales que abarcan nueve pasos formales:

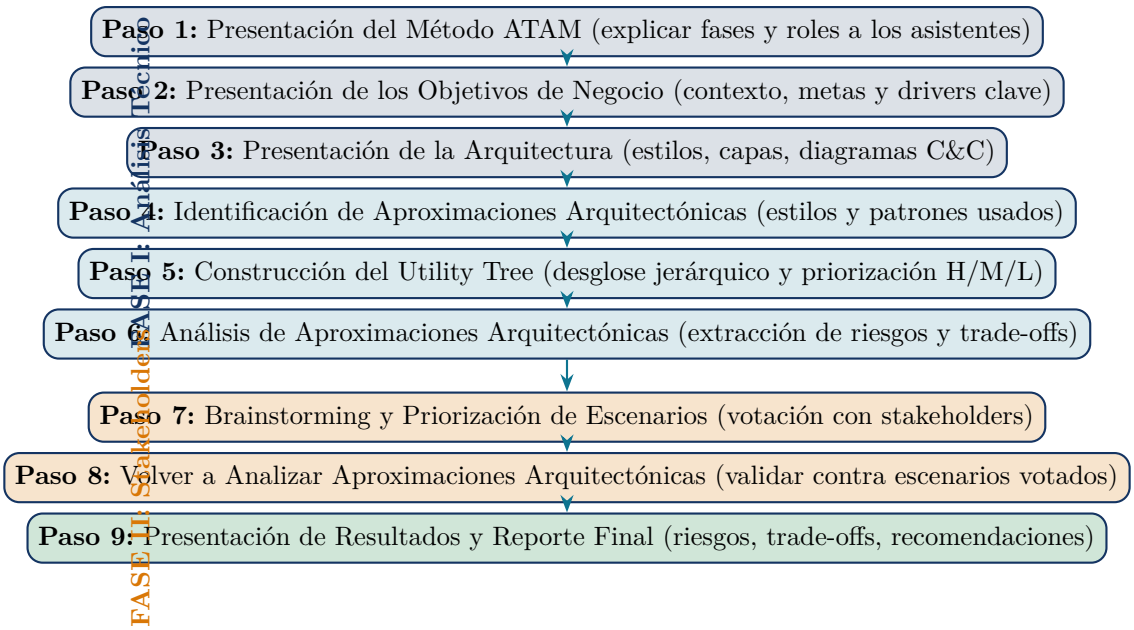



Figura 4.4: Secuencia de los nueve pasos formales de una evaluación ATAM.

▶ **Concepto Clave: Desglose Operacional de los 9 Pasos de ATAM**

1. **Paso 1: Presentación de ATAM:** El equipo evaluador explica la metodología, los objetivos, la agenda (típicamente 2 a 3 días de duración) y las reglas de discusión a los clientes y técnicos.
2. **Paso 2: Presentación de Objetivos de Negocio:** El cliente o gestor describe la misión comercial, el contexto del problema, las restricciones funcionales y los deseos de calidad primordiales.
3. **Paso 3: Presentación de la Arquitectura:** El arquitecto expone la solución técnica visualmente mediante vistas arquitectónicas, detallando restricciones hardware, middleware y estilos elegidos.
4. **Paso 4: Identificar Aproximaciones Arquitectónicas:** Se identifican los patrones y estilos adoptados (e.g., capas, colas de mensajes). Se exige analizar **al menos dos aproximaciones candidatas** para cada deseo de calidad clave.
5. **Paso 5: Construir el Utility Tree:** Se descomponen los atributos en sub-atributos y escenarios concretos cuantificables, clasificándolos en la matriz de prioridades $(H, M, L)^2$.
6. **Paso 6: Analizar Aproximaciones Arquitectónicas:** Se somete la arquitectura al estímulo de cada escenario prioritario, registrando formalmente los **Riesgos**, **No-Riesgos**, **Puntos Sensibles** y **Trade-offs**.
7. **Paso 7: Brainstorming y Priorización de Escenarios:** Se reúne al grupo ampliado de stakeholders. Se generan nuevos escenarios de uso y mediante un sistema de votación ponderada se eligen los escenarios más representativos.
8. **Paso 8: Volver a Analizar Aproximaciones:** Se contrastan los escenarios más votados en el Paso 7 contra la arquitectura para corroborar si descubren riesgos no detectados en la Fase I.
9. **Paso 9: Presentación de Resultados:** El equipo evaluador entrega el informe ejecutivo documentando todos los hallazgos, el árbol de utilidad final y la propuesta de cambios arquitectónicos necesarios.

## 4.6 Otros Métodos de Evaluación Arquitectónica

Aunque ATAM es el método más extendido, existen otros marcos metodológicos diseñados para contextos específicos:

- **SAAM (*Software Architecture Analysis Method*):** Fue el antecesor directo de ATAM desarrollado por Kazman et al. en 1994. Se enfocaba casi con exclusividad en la evaluación de la **Modificabilidad** y el coste de cambio.
- **ARID (*Active Reviews for Intermediate Designs*):** Diseñado para evaluar diseños parciales o subsistemas intermedios antes de completar la arquitectura global, combinando técnicas de revisión activa con escenarios de calidad.
- **CBAM (*Cost Benefit Analysis Method*):** Desarrollado por el SEI como extensión económica de ATAM. Modela matemáticamente el **Retorno de la Inversión (ROI)** de cada estrategia arquitectónica, ayudando a los directivos a decidir qué tácticas de calidad implementar en función de su coste en euros y su beneficio comercial derivado.
- **SARA (*Software Architecture Review and Assessment*):** Marco neutral que estandariza las directrices para la revisión de arquitecturas en consorcios industriales.

## 4.7 Ejercicios Resueltos y Casos Prácticos de Examen

★ **Ejercicio Resuelto: Construcción Completa de un Utility Tree para E-Commerce de Alta Demanda**

**Enunciado:**
Una plataforma de comercio electrónico debe rediseñarse para soportar la campaña de rebajas del

Black Friday. Los directores de negocio exigen:

1. Soportar picos masivos de navegación sin que la web se caiga.
2. Garantizar que las transacciones de cobro no superen nunca los 2 segundos.
3. Proteger los números de tarjeta de crédito según la normativa PCI-DSS.
4. Permitir cambiar de proveedor de envíos en menos de dos días de trabajo.

Construya el Árbol de Utilidad (*Utility Tree*) formal, desglosando atributos, sub-atributos, identificadores de escenario, partes afectadas del sistema y la matriz de prioridad ($Importancia, Dificultad$).

**Solución Razonada:**

| Atributo | Sub-atributo | Parte Afectada | Prioridad | Escenario Concreto |
|---|---|---|---|---|
| **Disponibilidad** | Tolerancia a fallos | Cluster Web / API | **(H, H)** | **SC-01:** Si un nodo del API Gateway cae bajo carga de 50,000 req/s, el balanceador redirige el tráfico al resto de nodos en $< 1$ s sin errores 500. |
| **Rendimiento** | Latencia de transacción | Servicio de Pagos | **(H, M)** | **SC-02:** Cuando 5,000 usuarios completan el pago simultáneamente, el 95 % de las transacciones finalizan en $< 2$ s. |
| **Seguridad** | Confidencialidad | Base de Datos | **(H, M)** | **SC-03:** Los datos de tarjetas se cifran en reposo con AES-256 (claves rotadas por KMS); un volcado no autorizado del disco no revela información legible. |
| **Modificabilidad** | Acoplamiento | Módulo Logística | **(M, L)** | **SC-04:** Reemplazar el adaptador de SEUR por DHL en tiempo de diseño exige modificar únicamente una clase adaptadora en $< 16$ horas-persona. |

### ★ Ejercicio Resuelto: Análisis ATAM: Identificación de Riesgos, Puntos Sensibles y Trade-offs

**Enunciado:**

El equipo de arquitectura del e-commerce presenta la siguiente propuesta técnica:

> *«Para garantizar que los pagos respondan en menos de 2 segundos, se configurará una caché distribuida en memoria (Redis) que almacenará el estado de las transacciones en vuelo sin persistir inmediatamente en el disco de la base de datos relacional PostgreSQL. Para máxima seguridad, cada transacción se firmará con una clave asimétrica RSA de 4096 bits».*

Como evaluador técnico ATAM, identifique de forma justificada los riesgos, puntos sensibles y trade-offs de esta propuesta.

**Solución Razonada:**

1. **Riesgo (*Risk*):**
   Almacenar transacciones en vuelo únicamente en la caché en memoria de Redis sin escritura síncrona en PostgreSQL supone un **riesgo crítico de disponibilidad e integridad**: ante un fallo

eléctrico repentino o reinicio del contenedor Redis, se perderán compras de clientes ya cobradas en el banco, violando la consistencia transaccional.

2. **Punto de Sensibilidad (*Sensitivity Point*):**
   El tiempo de cálculo de la firma digital RSA de 4096 bits es un **punto de sensibilidad clave para el rendimiento global**: si el volumen de peticiones supera los 500 pagos/s, la sobrecarga criptográfica de CPU colapsará el servicio, degradando la latencia mucho más allá de los 2 segundos requeridos.
3. **Compromiso (*Trade-off*):**
   **Seguridad vs. Rendimiento vs. Coste:** Aumentar el tamaño de clave RSA a 4096 bits maximiza la seguridad frente a ataques criptográficos (*Security*), pero degrada de manera severa el tiempo de respuesta (*Performance*) o bien obliga a comprar hardware de aceleración criptográfica HSM con altísimo coste financiero (*Cost*).

## 4.8 Resumen Ejecutivo y Preguntas de Autoevaluación

**Resumen de Conceptos Clave del Tema 4**

- **Atributos de Calidad (QAs):** Son los requisitos no funcionales que moldean la forma estructural de la arquitectura. Deben expresarse siempre mediante valores cuantitativos verificables.
- **ISO/IEC 25010:** Estándar internacional que categoriza la calidad del software. La edición 2023 incorpora formalmente la dimensión de *Safety* (seguridad de vidas y entorno).
- **Escenarios de Calidad (SEI):** Estructurados obligatoriamente en 6 partes: *Fuente, Estímulo, Entorno, Artefacto, Respuesta y Medida de la Respuesta.*
- **Utility Tree:** Descompone las metas de negocio en escenarios concretos y los prioriza mediante la matriz $(Importancia, Dificultad) \in \{H, M, L\}^2$, centrando el análisis en los cuadrantes $(H, H)$ y $(H, M)$.
- **Método ATAM:** Método de evaluación por excelencia. Sus cuatro hallazgos cardinales son: **Riesgos**, **No-Riesgos**, **Puntos de Sensibilidad** y **Compromisos (*Trade-offs*)**.
- **Fases de ATAM:** Fase I (pasos 1-6 con equipo técnico reducido) y Fase II (pasos 7-9 con grupo ampliado de stakeholders para votación y generación de reporte final).

### 4.8.1 Batería de Preguntas de Autoevaluación

1. **¿Por qué un Requisito No Funcional puede aplicarse múltiples veces en el diseño mientras que un Requisito Funcional solo se utiliza una vez?**
   *Respuesta orientativa:* Un requisito funcional describe una operación específica que se confina en un caso de uso y en una clase determinada. En cambio, un atributo de calidad como la seguridad o la latencia afecta transversalmente a la autenticación, a la base de datos, a las comunicaciones y a los logs, debiendo considerarse en cada decisión estructural.

2. **¿Cuáles son las seis partes canónicas de un Escenario de Calidad según el SEI?**
   *Respuesta orientativa:* 1. Fuente del estímulo; 2. Estímulo; 3. Entorno; 4. Artefacto afectado; 5. Respuesta; 6. Medida cuantitativa de la respuesta.

3. **¿Cuál es la diferencia técnica entre un Punto de Sensibilidad y un Trade-off en ATAM?**
   *Respuesta orientativa:* Un punto de sensibilidad es un parámetro o componente en el que un pequeño cambio produce una alteración drástica en **un único** atributo de calidad. Un trade-off es una decisión arquitectónica que afecta simultáneamente a **dos o más atributos contrapuestos**, mejorando uno a expensas de empeorar otro.

4. **¿Por qué los escenarios clasificados como $(L, L)$ se descartan del análisis profundo en un taller ATAM?**

*Respuesta orientativa:* Porque representan requisitos de bajo impacto comercial para los stakeholders y de muy baja dificultad técnica para los ingenieros. Dedicar tiempo de evaluación a estos escenarios desperdiciaría recursos que deben consagrarse a mitigar los riesgos $(H, H)$ que pueden hacer fracasar el proyecto.